

Rachel Hardy

ST 563 (601)

Final Project

Contents

Data Preparation	2
Data Exploration	3
Data Splitting	9
Data Analysis	9
(Section 1) Linear Regression Models	10
(Section 2) Nonparametric Regression Models	24
(Section 3) Regression Tree Models	33
(Section 4) Stacking Models	41
Summaries	44

Data Preparation

Below we will prepare the data!

```
## Original Data ----
bleaching_data <- fread("global_bleaching_environmental.csv",
                        na.strings = c("", "NA", "nd"))

# Using only data from the FRRP source
# Removing observations where the response variable is missing
final_data1 <- bleaching_data %>%
  filter(!is.na(Percent_Bleaching)) %>%
  filter(Data_Source == "FRRP") %>%
  distinct(Site_ID, Sample_ID, .keep_all = TRUE)

#Removing unwanted variables
final_data2 <- final_data1 %>%
  subset(select = -c(Site_ID, Sample_ID, Data_Source, Latitude_Degrees,
                    Longitude_Degrees, Ocean_Name, Reef_ID,
                    Realm_Name, Ecoregion_Name, Country_Name,
                    State_Island_Province_Name, City_Town_Name,
                    Site_Name, Date_Day, Date_Month, Date_Year,
                    Substrate_Name, Percent_Cover,
                    Bleaching_Level, Temperature_Mean,
                    Temperature_Minimum, Temperature_Maximum,
                    Temperature_Kelvin_Standard_Deviation,
                    SSTA_Standard_Deviation, SSTA_Mean,
                    SSTA_Minimum, SSTA_Maximum, SSTA_Frequency,
                    SSTA_Frequency_Standard_Deviation, SSTA_FrequencyMax,
                    SSTA_FrequencyMean, SSTA_DHW_Standard_Deviation,
                    SSTA_DHWMax, SSTA_DHWMean, TSA_Standard_Deviation,
                    TSA_Minimum, TSA_Maximum, TSA_Mean, TSA_Frequency,
                    TSA_Frequency_Standard_Deviation, TSA_FrequencyMax,
                    TSA_FrequencyMean, TSA_DHW_Standard_Deviation,
                    TSA_DHWMax, TSA_DHWMean, Date, Site_Comments,
                    Sample_Comments, Bleaching_Comments))

# Final data set
final_data <- final_data2 %>%
  dplyr::select(
    Distance_to_Shore,
    Turbidity,
    Cyclone_Frequency,
    Depth_m,
    Windspeed,
    Temperature_Kelvin,
    ClimSST,
    SSTA,
    SSTA_DHW,
    TSA,
    TSA_DHW,
    Exposure,
    Percent_Bleaching
  )
```

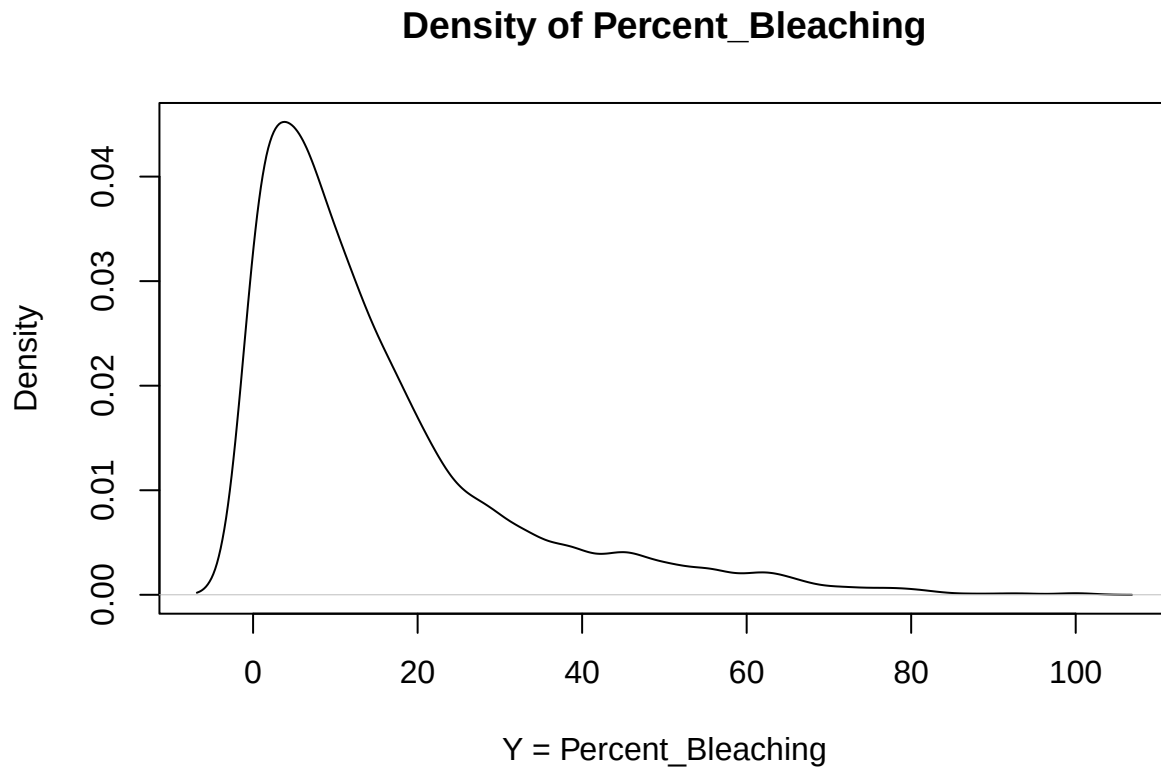
```
final_data$Exposure <- as.factor(final_data$Exposure)
levels(final_data$Exposure)
```

```
## [1] "Exposed"    "Sheltered"
```

Data Exploration

Below we will explore the data!

```
# Density plot of Percent_Bleaching
plot(density(final_data$Percent_Bleaching),
     main = "Density of Percent_Bleaching",
     xlab = "Y = Percent_Bleaching")
```



```
# Looking at the correlations of all numeric variables
cor(final_data[, -c(12,13)])
```

```
##           Distance_to_Shore  Turbidity Cyclone_Frequency
## Distance_to_Shore      1.0000000000  0.03221876      -0.018151069
## Turbidity              0.0322187589  1.00000000      0.074413157
## Cyclone_Frequency     -0.0181510692  0.07441316      1.000000000
## Depth_m               0.0001583411 -0.52241006     -0.193159737
```

```

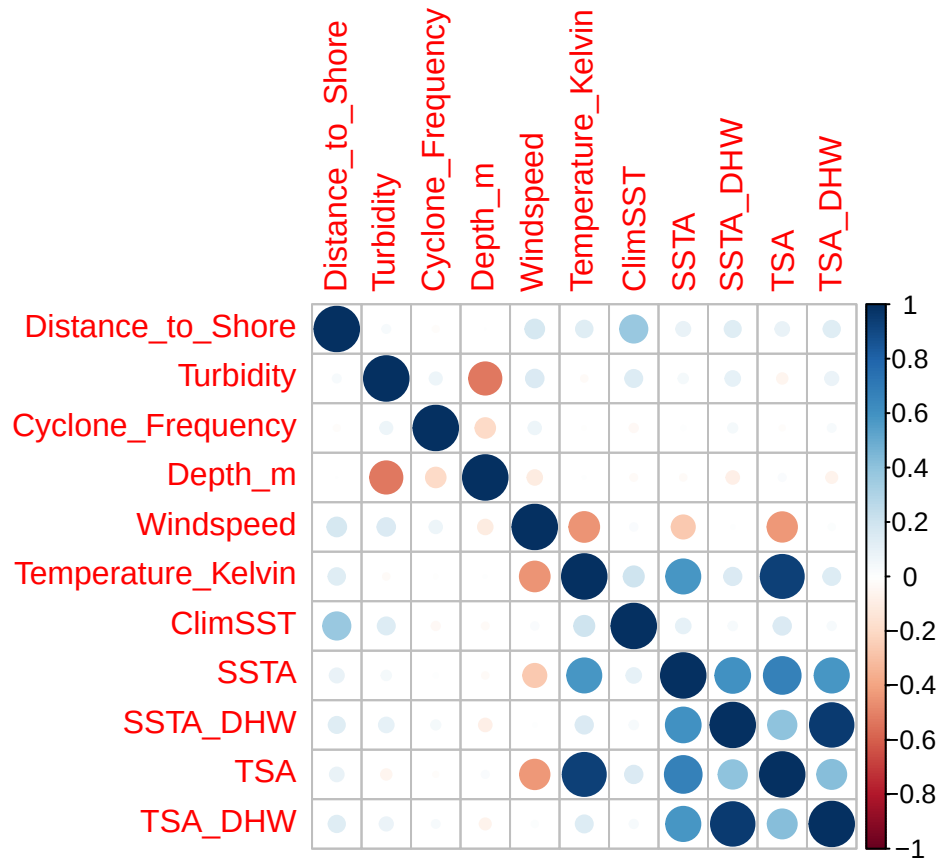
## Windspeed          0.1774254540  0.15583686      0.078297845
## Temperature_Kelvin  0.1377140148 -0.02130741     -0.005882463
## ClimSST            0.3730675931  0.14546089     -0.034820621
## SSTA               0.0994044078  0.04567915      0.008839850
## SSTA_DHW           0.1300745414  0.10858516      0.043145118
## TSA                0.0947075381 -0.05506677     -0.013230329
## TSA_DHW            0.1340568585  0.08804628      0.031243155
##
##          Depth_m    Windspeed Temperature_Kelvin    ClimSST
## Distance_to_Shore  0.0001583411  0.177425454    0.137714015  0.37306759
## Turbidity          -0.5224100553  0.155836862    -0.021307415  0.14546089
## Cyclone_Frequency -0.1931597371  0.078297845    -0.005882463 -0.03482062
## Depth_m            1.0000000000 -0.106781126    0.003750874 -0.02398784
## Windspeed          -0.1067811257  1.000000000    -0.445024529  0.02855445
## Temperature_Kelvin  0.0037508744 -0.445024529    1.000000000  0.20313453
## ClimSST            -0.0239878362  0.028554446    0.203134531  1.00000000
## SSTA               -0.0213843831 -0.268837871    0.583704243  0.10844314
## SSTA_DHW           -0.0804860037  0.005872296    0.150019770  0.03728247
## TSA                0.0254569757 -0.434518725    0.931878092  0.15252589
## TSA_DHW            -0.0615352974  0.019541016    0.144969188  0.03421778
##
##          SSTA    SSTA_DHW    TSA    TSA_DHW
## Distance_to_Shore  0.09940441  0.130074541  0.09470754  0.13405686
## Turbidity          0.04567915  0.108585157 -0.05506677  0.08804628
## Cyclone_Frequency  0.00883985  0.043145118 -0.01323033  0.03124315
## Depth_m            -0.02138438 -0.080486004  0.02545698 -0.06153530
## Windspeed          -0.26883787  0.005872296 -0.43451873  0.01954102
## Temperature_Kelvin  0.58370424  0.150019770  0.93187809  0.14496919
## ClimSST            0.10844314  0.037282473  0.15252589  0.03421778
## SSTA               1.00000000  0.607784503  0.67821459  0.58768466
## SSTA_DHW           0.60778450  1.000000000  0.40354169  0.95247543
## TSA                0.67821459  0.403541693  1.00000000  0.42256161
## TSA_DHW            0.58768466  0.952475426  0.42256161  1.00000000

```

```

# Correlation plot
corrplot::corrplot(corr = cor(final_data[, -c(12,13)]))

```

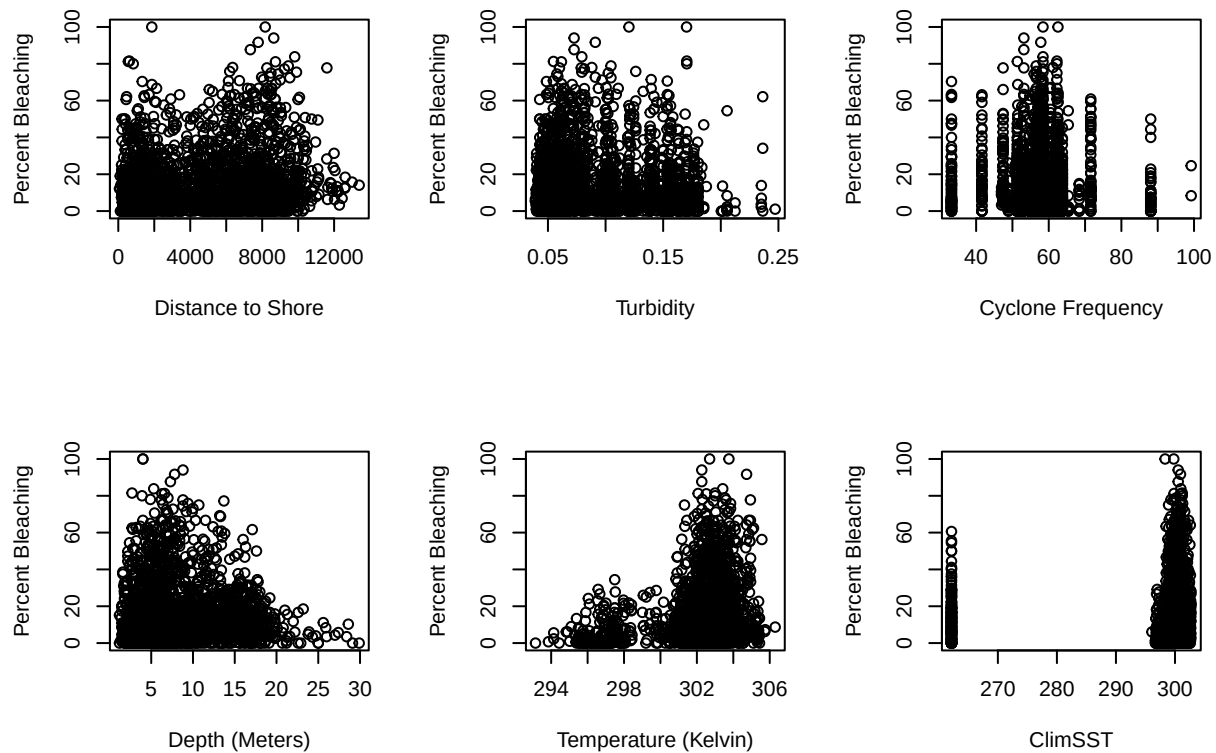


Below we will investigate the correlation of each variable with the response variable by using scatter plots and box plots.

```
par(mfrow = c(2,3))

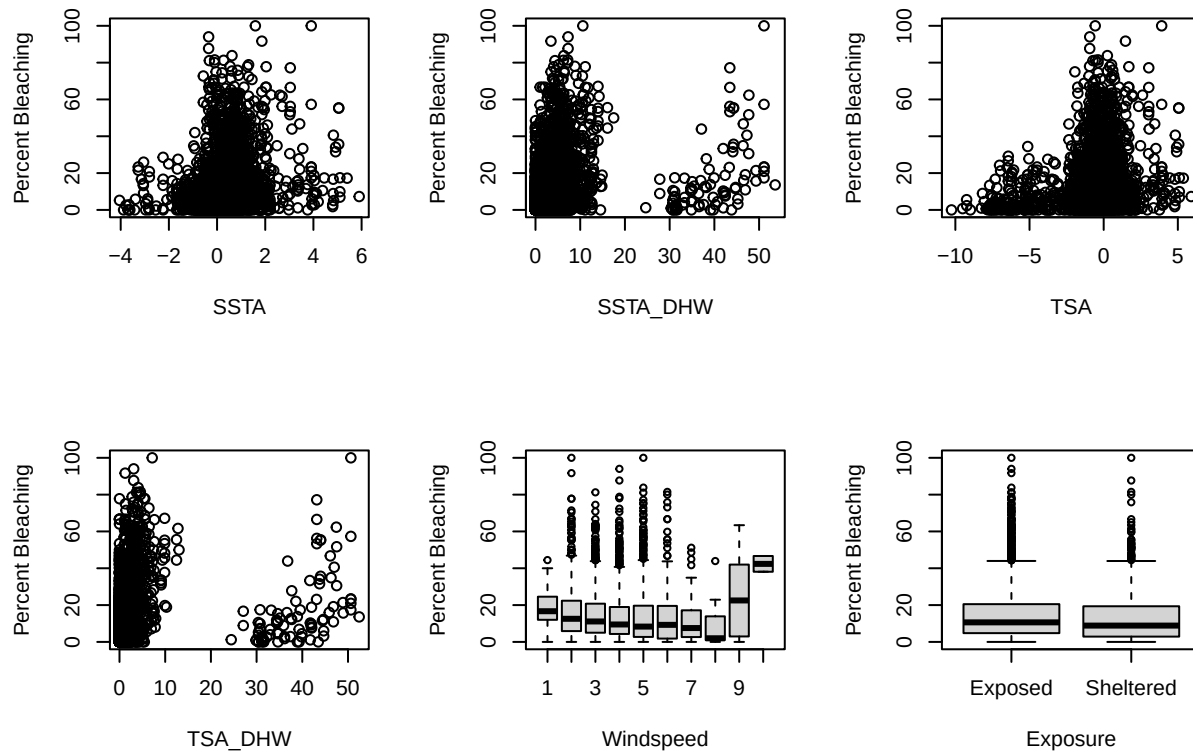
#First set of graphs
plot(x = final_data$Distance_to_Shore,
     y = final_data$Percent_Bleaching,
     xlab = "Distance to Shore",
     ylab = "Percent Bleaching")
plot(x = final_data$Turbidity,
     y = final_data$Percent_Bleaching,
     xlab = "Turbidity",
     ylab = "Percent Bleaching")
plot(x = final_data$Cyclone_Frequency,
     y = final_data$Percent_Bleaching,
     xlab = "Cyclone Frequency",
     ylab = "Percent Bleaching")
plot(x = final_data$Depth_m,
     y = final_data$Percent_Bleaching,
     xlab = "Depth (Meters)",
     ylab = "Percent Bleaching")
plot(x = final_data$Temperature_Kelvin,
     y = final_data$Percent_Bleaching,
     xlab = "Temperature (Kelvin)",
     ylab = "Percent Bleaching")
```

```
plot(x = final_data$ClimSST,
     y = final_data$Percent_Bleaching,
     xlab = "ClimSST",
     ylab = "Percent Bleaching")
```



```
# Second set of graphs
plot(x = final_data$SSTA,
     y = final_data$Percent_Bleaching,
     xlab = "SSTA",
     ylab = "Percent Bleaching")
plot(x = final_data$SSTA_DHW,
     y = final_data$Percent_Bleaching,
     xlab = "SSTA_DHW",
     ylab = "Percent Bleaching")
plot(x = final_data$TSA,
     y = final_data$Percent_Bleaching,
     xlab = "TSA",
     ylab = "Percent Bleaching")
plot(x = final_data$TSA_DHW,
     y = final_data$Percent_Bleaching,
     xlab = "TSA_DHW",
     ylab = "Percent Bleaching")
boxplot(Percent_Bleaching ~ Windspeed,
        data = final_data,
        ylab = "Percent Bleaching")
```

```
boxplot(Percent_Bleaching ~ Exposure,
        data = final_data,
        ylab = "Percent Bleaching")
```

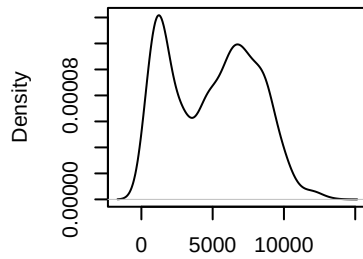


- We will remove TSA because it is highly correlated with `Temperature_Kelvin`, having a correlation of 0.932.
- We will remove TSA_DHW because it is highly correlated with `SSTA_DHW`, having a correlation of 0.952.

```
par(mfrow = c(2,3))

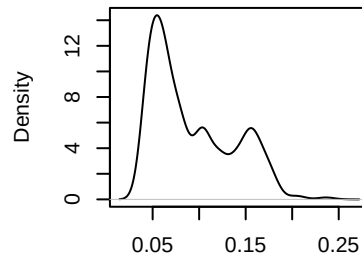
plot(density(final_data$Distance_to_Shore),
     main = "Density of Distance_to_Shore")
plot(density(final_data$Turbidity),
     main = "Density of Turbidity")
plot(density(final_data$Cyclone_Frequency),
     main = "Density of Cyclone_Frequency")
plot(density(final_data$Depth_m),
     main = "Density of Depth_m")
plot(density(final_data$Windspeed),
     main = "Density of Windspeed")
plot(density(final_data$Temperature_Kelvin),
     main = "Density of Temperature_Kelvin")
```

Density of Distance_to_Shore



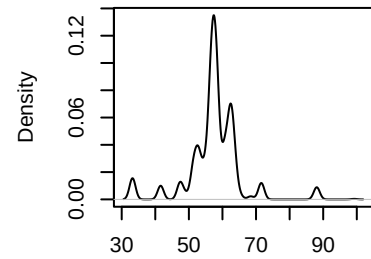
N = 2394 Bandwidth = 581.2

Density of Turbidity



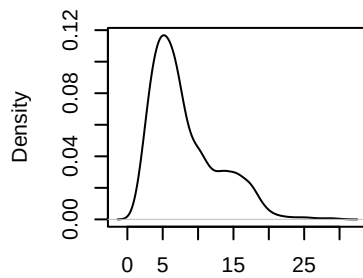
N = 2394 Bandwidth = 0.008269

Density of Cyclone_Frequency



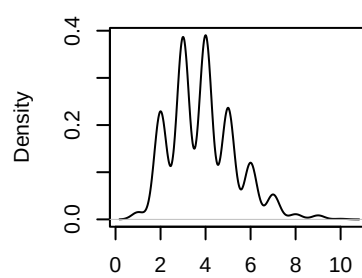
N = 2394 Bandwidth = 0.8954

Density of Depth_m



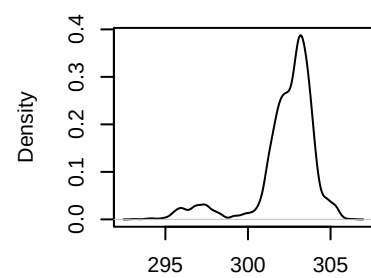
N = 2394 Bandwidth = 0.8643

Density of Windspeed



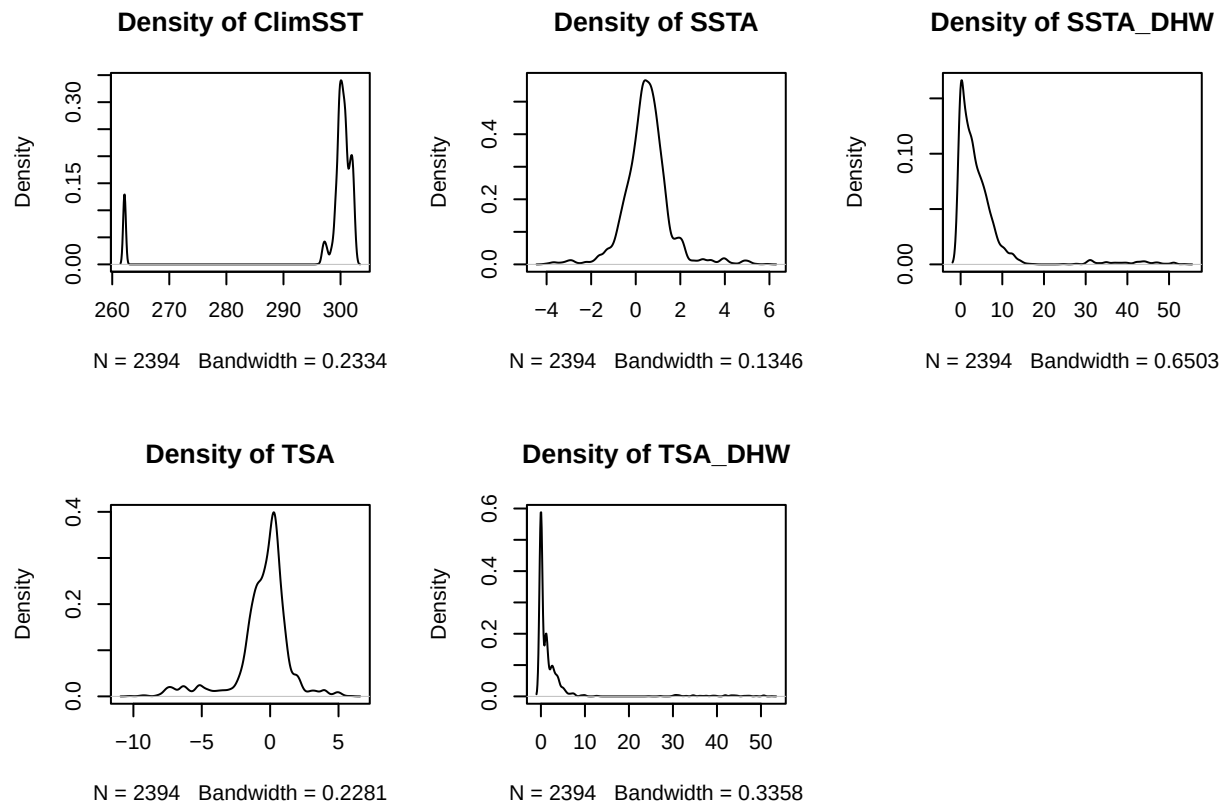
N = 2394 Bandwidth = 0.275

Density of Temperature_Kelvin



N = 2394 Bandwidth = 0.2253

```
plot(density(final_data$ClimSST),  
     main = "Density of ClimSST")  
plot(density(final_data$SSTA),  
     main = "Density of SSTA")  
plot(density(final_data$SSTA_DHW),  
     main = "Density of SSTA_DHW")  
plot(density(final_data$TSA),  
     main = "Density of TSA")  
plot(density(final_data$TSA_DHW),  
     main = "Density of TSA_DHW")
```



Data Splitting

Below we will split the data!

```
# Remove more unwanted variables
coral_data <- final_data %>%
  subset(select = -c(TSA, TSA_DHW))

set.seed(123)

# Splitting the data into training and test sets
index <- createDataPartition(coral_data$Percent_Bleaching,
                             p = 0.75,
                             list = FALSE,
                             times = 1)

train <- coral_data[index, ]
test <- coral_data[-index, ]
```

Data Analysis

Below we will model the data!

(Section 1) Linear Regression Models

Model 1.1: Multiple Linear Regression (MLR)

Below we will have two different models, a regression model without interaction terms and a regression model with interaction terms.

- Model 1.1a: Multiple Linear Regression (without interaction terms)
- Model 1.1b: Multiple Linear Regression (with interaction terms)

Model 1.1a: Multiple Linear Regression (without interaction terms) Below is a multiple linear regression model without interaction terms.

```
##### Model 1.1a #####  
# Model 1.1a: Multiple Linear Regression (without interaction terms)
```

```
set.seed(123)
```

```
# Fitting the model
```

```
lm_model1 <- lm(Percent_Bleaching ~ .,  
                data = train)
```

```
summary(lm_model1)
```

```
##  
## Call:  
## lm(formula = Percent_Bleaching ~ ., data = train)  
##  
## Residuals:  
##      Min       1Q   Median       3Q      Max   
## -32.111 -10.167  -4.058   4.872  87.209   
##  
## Coefficients:  
##              Estimate Std. Error t value Pr(>|t|)      
## (Intercept)   -6.285e+02  8.258e+01  -7.611 4.37e-14 ***  
## Distance_to_Shore  3.807e-04  1.421e-04   2.679 0.007442 **  
## Turbidity       -4.431e+01  1.205e+01  -3.677 0.000243 ***  
## Cyclone_Frequency -1.434e-01  4.629e-02  -3.098 0.001977 **  
## Depth_m        -2.516e-01  9.157e-02  -2.748 0.006059 **  
## Windspeed       3.675e-01  2.994e-01   1.228 0.219783   
## Temperature_Kelvin 2.210e+00  2.745e-01   8.050 1.49e-15 ***  
## ClimSST        -4.926e-02  3.947e-02  -1.248 0.212093   
## SSTA          -1.652e+00  5.897e-01  -2.802 0.005141 **  
## SSTA_DHW       4.538e-01  6.299e-02   7.203 8.62e-13 ***  
## ExposureSheltered 1.044e+00  1.183e+00   0.882 0.377895   
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## Residual standard error: 15.41 on 1787 degrees of freedom  
## Multiple R-squared:  0.09748,    Adjusted R-squared:  0.09243   
## F-statistic: 19.3 on 10 and 1787 DF,  p-value: < 2.2e-16
```

We can see above that the following variables are NOT significant:

- Windspeed
- ClimSST
- Exposure

Below we will recreate the model without these variables.

```
# Fitting the model
lm_model1_final <- lm(Percent_Bleaching ~ Distance_to_Shore +
  Turbidity +
  Cyclone_Frequency +
  Depth_m +
  Temperature_Kelvin +
  SSTA +
  SSTA_DHW,
  data = train)
summary(lm_model1_final)

##
## Call:
## lm(formula = Percent_Bleaching ~ Distance_to_Shore + Turbidity +
##     Cyclone_Frequency + Depth_m + Temperature_Kelvin + SSTA +
##     SSTA_DHW, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -31.897 -10.163  -3.972   5.020  86.689
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -5.937e+02  7.664e+01  -7.747 1.56e-14 ***
## Distance_to_Shore  3.319e-04  1.222e-04   2.717  0.00665 **
## Turbidity        -3.917e+01  9.859e+00  -3.973 7.39e-05 ***
## Cyclone_Frequency -1.354e-01  4.611e-02  -2.937  0.00335 **
## Depth_m          -2.680e-01  9.112e-02  -2.941  0.00331 **
## Temperature_Kelvin 2.050e+00  2.538e-01   8.077 1.21e-15 ***
## SSTA             -1.702e+00  5.889e-01  -2.890  0.00390 **
## SSTA_DHW          4.616e-01  6.279e-02   7.351 2.98e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 15.42 on 1790 degrees of freedom
## Multiple R-squared:  0.09552,    Adjusted R-squared:  0.09199
## F-statistic: 27.01 on 7 and 1790 DF,  p-value: < 2.2e-16

# Predict response for the test (holdout) set
pred <- predict(lm_model1_final, newdata = test)

# Compute test MSE
lm_error1 <- mean((test$Percent_Bleaching - pred)**2)
paste("Our estimated test MSE is", round(lm_error1, 2))

## [1] "Our estimated test MSE is 216.46"
```

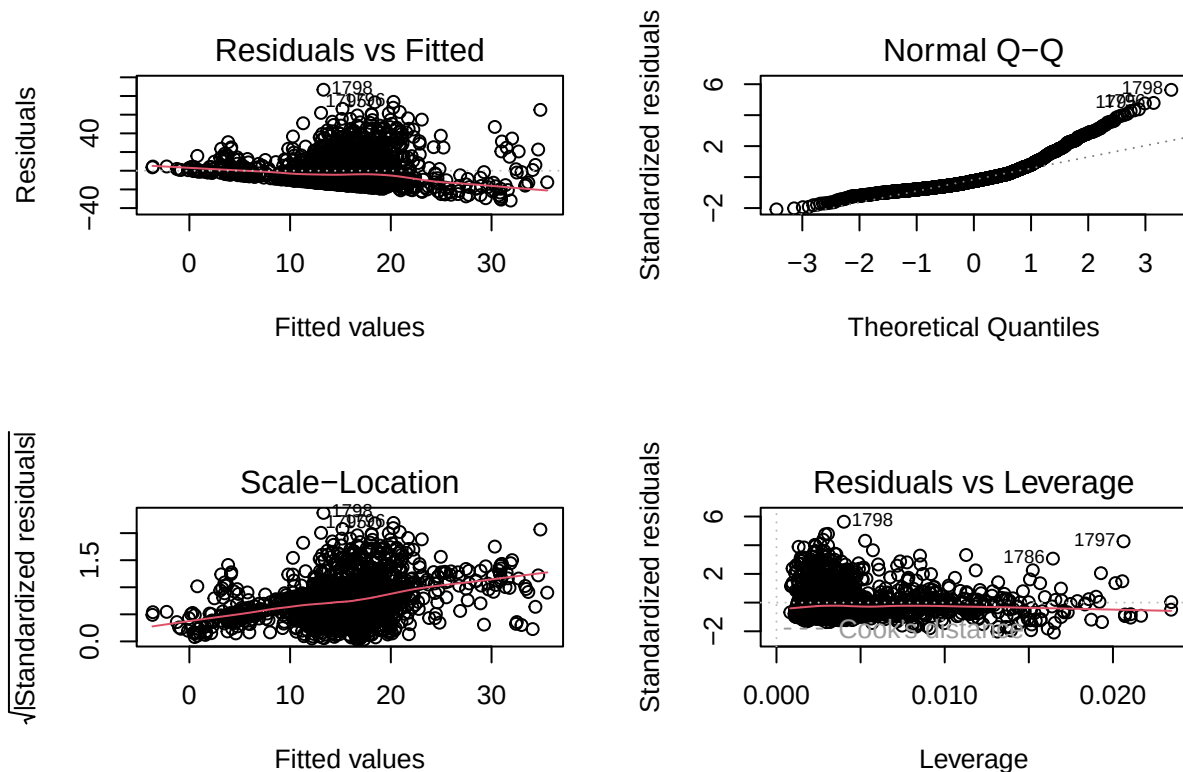
```
# Compute test RMSE
lm_rmse1 <- sqrt(lm_error1)
paste("Our estimated test RMSE is", round(lm_rmse1, 2))
```

```
## [1] "Our estimated test RMSE is 14.71"
```

```
# Compute R_squared
r_squared_lm_model1 <- round(cor(pred, test$Percent_Bleaching)**2, 4)
paste("Our estimated R_squared is", round(r_squared_lm_model1, 4))
```

```
## [1] "Our estimated R_squared is 0.0862"
```

```
par(mfrow = c(2, 2))
plot(lm_model1_final)
```



Model 1.1b: Multiple Linear Regression (with interaction terms) Below is a multiple linear regression model with interaction terms.

```
##### Model 1.1b #####
# Model 1.1b: Multiple Linear Regression (with interaction terms)

set.seed(123)
```

```

# Fitting the model
lm_model2 <- lm(Percent_Bleaching ~ (.)^2,
               data = train)
#summary(lm_model2)

lm_model2_final <- lm(Percent_Bleaching ~ Distance_to_Shore*Temperature_Kelvin +
                     Distance_to_Shore*ClimSST +
                     Turbidity*Depth_m +
                     Cyclone_Frequency*Windspeed +
                     Depth_m*Temperature_Kelvin +
                     Temperature_Kelvin*SSTA_DHW +
                     ClimSST*SSTA_DHW +
                     SSTA*SSTA_DHW,
                     data = train)
summary(lm_model2_final)

##
## Call:
## lm(formula = Percent_Bleaching ~ Distance_to_Shore * Temperature_Kelvin +
##     Distance_to_Shore * ClimSST + Turbidity * Depth_m + Cyclone_Frequency *
##     Windspeed + Depth_m * Temperature_Kelvin + Temperature_Kelvin *
##     SSTA_DHW + ClimSST * SSTA_DHW + SSTA * SSTA_DHW, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -44.589  -8.958  -3.276   5.389  85.278
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -1.007e+02  1.565e+02  -0.644  0.51980
## Distance_to_Shore    -3.123e-02  2.091e-02  -1.493  0.13552
## Temperature_Kelvin    3.426e-01  5.227e-01   0.655  0.51224
## ClimSST           1.457e-01  6.804e-02   2.141  0.03242 *
## Turbidity        -7.874e+01  1.797e+01  -4.382 1.24e-05 ***
## Depth_m           2.157e+01  1.259e+01   1.714  0.08675 .
## Cyclone_Frequency   -5.452e-01  1.375e-01  -3.966 7.60e-05 ***
## Windspeed         -6.279e+00  2.181e+00  -2.879  0.00403 **
## SSTA_DHW          -1.061e+02  1.638e+01  -6.481 1.18e-10 ***
## SSTA              5.636e-01  6.308e-01   0.894  0.37169
## Distance_to_Shore:Temperature_Kelvin  2.117e-04  6.804e-05   3.112  0.00189 **
## Distance_to_Shore:ClimSST    -1.081e-04  3.922e-05  -2.757  0.00589 **
## Turbidity:Depth_m           6.090e+00  2.828e+00   2.153  0.03143 *
## Cyclone_Frequency:Windspeed  1.183e-01  3.742e-02   3.162  0.00159 **
## Temperature_Kelvin:Depth_m   -7.354e-02  4.167e-02  -1.765  0.07776 .
## Temperature_Kelvin:SSTA_DHW    3.706e-01  5.353e-02   6.923 6.15e-12 ***
## ClimSST:SSTA_DHW    -1.422e-02  1.064e-02  -1.337  0.18149
## SSTA_DHW:SSTA    -5.297e-01  4.770e-02 -11.106 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 14.81 on 1780 degrees of freedom
## Multiple R-squared:  0.1705, Adjusted R-squared:  0.1626
## F-statistic: 21.52 on 17 and 1780 DF, p-value: < 2.2e-16

```

```
# Predict response for the test (holdout) set
pred <- predict(lm_model2_final, newdata = test)

# Compute test MSE
lm_error2 <- mean((test$Percent_Bleaching - pred)**2)
paste("Our estimated test MSE is", round(lm_error2, 2))
```

```
## [1] "Our estimated test MSE is 206.77"
```

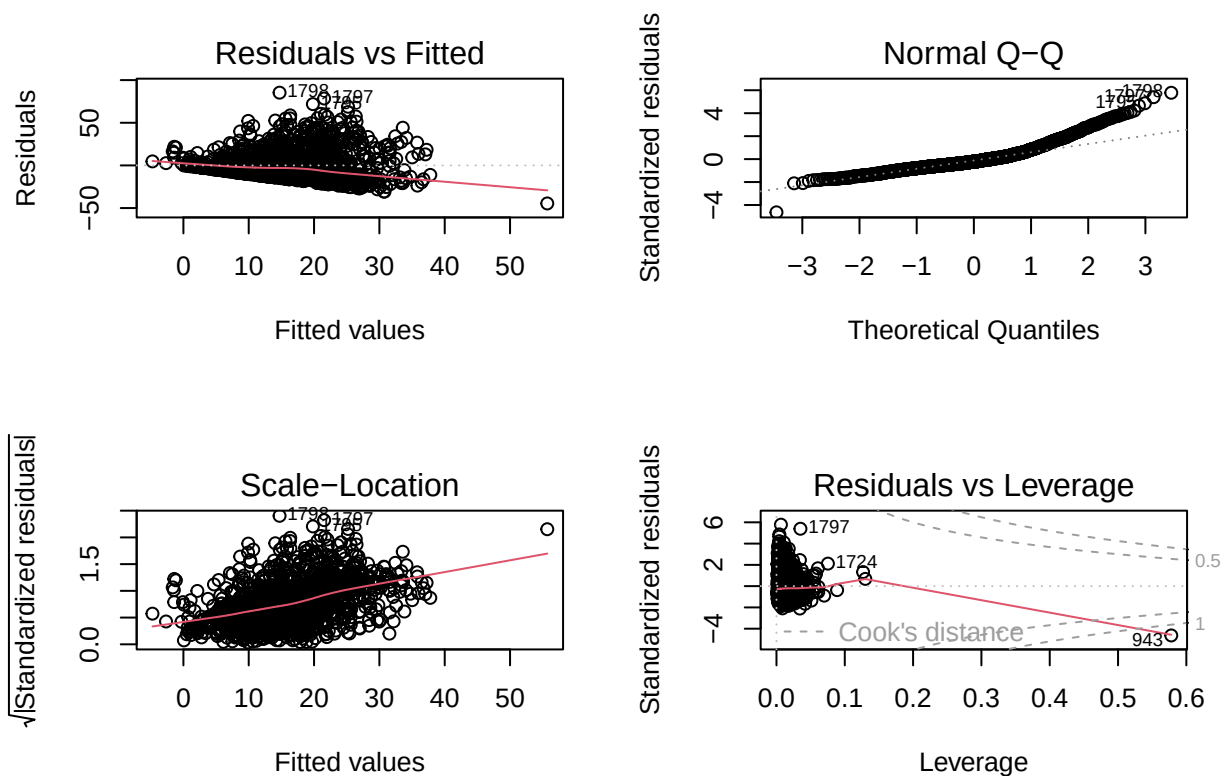
```
# Compute test RMSE
lm_rmse2 <- sqrt(lm_error2)
paste("Our estimated test RMSE is", round(lm_rmse2, 2))
```

```
## [1] "Our estimated test RMSE is 14.38"
```

```
# Compute R_squared
r_squared_lm_model2 <- round(cor(pred, test$Percent_Bleaching)**2, 4)
paste("Our estimated R_squared is", round(r_squared_lm_model2, 4))
```

```
## [1] "Our estimated R_squared is 0.1339"
```

```
par(mfrow = c(2, 2))
plot(lm_model2_final)
```



Model 1.2: Subset Selection

Note that all of the three subset selection models chose models of size 7 with the following variables:

- Distance_to_Shore
- Turbidity
- Cyclone_Frequency
- Depth_m
- Temperature_Kelvin
- SSTA
- SSTA_DHW

Windspeed, ClimSST, and Exposure were not included in any of the models.

Note that we already fit a linear model with these 7 variables above. It is Model 1.1a.

Model 1.2a: Best Subset Selection Below we will execute best subset selection.

```
##### Model 1.2a #####
# Model 1.2a: Best Subset Selection

set.seed(123)

# Fitting the model
best_subset <- regsubsets(Percent_Bleaching ~ .,
                          data = train,
                          nvmax = 10)

# Model summary
summary_best_subset <- summary(best_subset)

# Creating a data frame of performance metrics
metrics_best <- data.frame(model_size = seq(from = 1, to = 10, by = 1),
                           aic = summary_best_subset$aic,
                           bic = summary_best_subset$bic,
                           adjR2 = summary_best_subset$adjr2)

metrics_best

##      model_size      aic      bic      adjR2
## 1             1 90.347904 -74.0415 0.04778049
## 2             2 38.777408 -118.4034 0.07433592
## 3             3 31.547511 -119.9966 0.07848930
## 4             4 24.397483 -121.5566 0.08260688
## 5             5 19.233264 -121.1836 0.08572332
## 6             6 14.251140 -120.6568 0.08875097
## 7             7  8.866232 -120.5615 0.09198623
## 8             8  9.131293 -114.8097 0.09235882
## 9             9  9.777926 -108.6759 0.09253815
## 10            10 11.000000 -101.9640 0.09242543

# Finding which model has the lowest Cp and BIC
c("Lowest Cp" = which.min(summary_best_subset$aic),
  "Lowest BIC" = which.min(summary_best_subset$bic),
  "Highest Adj R2" = which.max(summary_best_subset$adjr2))
```

```
##      Lowest Cp      Lowest BIC Highest Adj R2
##      7              4              9
```

We will use a model of size 7 because it has a higher R^2 than the model of size 4.

```
set.seed(123)

# Fitting the model
best_subset_final <- regsubsets(Percent_Bleaching ~ .,
                                data = test,
                                nvmax = 10)

# Model summary
summary_best_subset_final <- summary(best_subset_final)

# Reporting coefficients of best model
coef(best_subset_final, 7)
```

```
##      (Intercept) Distance_to_Shore      Turbidity Cyclone_Frequency
##      -5.825333e+02  2.664414e-04      -2.898639e+01      -8.067351e-02
##      Depth_m Temperature_Kelvin      SSTA      SSTA_DHW
##      -1.728421e-01  1.997514e+00      -2.608972e+00      5.374973e-01
```

Model 1.2b: Forward Stepwise Selection Below we will execute forward stepwise selection.

```
##### Model 1.2b #####
# Model 1.2b: Forward Stepwise Selection
```

```
set.seed(123)

# Fitting the model
forward_subset <- regsubsets(Percent_Bleaching ~ .,
                             data = train,
                             nvmax = 10,
                             method = "forward")

# Model summary
summary_forward <- summary(forward_subset)

# Creating a data frame of performance metrics
metrics_forward <- data.frame(model_size = seq(from = 1, to = 10, by = 1),
                              aic = summary_forward$cp,
                              bic = summary_forward$bic,
                              adjR2 = summary_forward$adjr2)

metrics_forward
```

```
##      model_size      aic      bic      adjR2
## 1           1  90.347904 -74.0415  0.04778049
## 2           2  38.777408 -118.4034  0.07433592
## 3           3  31.547511 -119.9966  0.07848930
## 4           4  24.397483 -121.5566  0.08260688
## 5           5  19.233264 -121.1836  0.08572332
```

```
## 6          6 15.498116 -119.4104 0.08811908
## 7          7  8.866232 -120.5615 0.09198623
## 8          8  9.131293 -114.8097 0.09235882
## 9          9  9.777926 -108.6759 0.09253815
## 10         10 11.000000 -101.9640 0.09242543
```

```
# Finding which model has the lowest Cp and BIC
c("Lowest Cp" = which.min(summary_forward$cp),
  "Lowest BIC" = which.min(summary_forward$bic),
  "Highest Adj R2" = which.max(summary_forward$adjr2))
```

```
##      Lowest Cp      Lowest BIC Highest Adj R2
##          7          4          9
```

We will use a model of size 7 because it has a higher R^2 than the model of size 4.

```
set.seed(123)

# Fitting the model
forward_final <- regsubsets(Percent_Bleaching ~ .,
                           data = test,
                           nvmax = 10,
                           method = "forward")

# Model summary
summary_forward_final <- summary(forward_final)

# Reporting coefficients of best model
coef(forward_final, 7)
```

```
##      (Intercept) Distance_to_Shore      Turbidity Cyclone_Frequency
##      -5.825333e+02      2.664414e-04      -2.898639e+01      -8.067351e-02
##      Depth_m Temperature_Kelvin      SSTA      SSTA_DHW
##      -1.728421e-01      1.997514e+00      -2.608972e+00      5.374973e-01
```

Model 1.2c: Backward Stepwise Selection Below we will execute backward stepwise selection.

```
##### Model 1.2c #####
# Model 1.2c: Backward Stepwise Selection

set.seed(123)

# Fitting the model
backward_subset <- regsubsets(Percent_Bleaching ~ .,
                             data = train,
                             nvmax = 10,
                             method = "backward")

# Model summary
summary_backward <- summary(backward_subset)

# Creating a data frame of performance metrics
```

```
metrics_backward <- data.frame(model_size = seq(from = 1, to = 10, by = 1),
                               aic = summary_backward$cp,
                               bic = summary_backward$bic,
                               adjR2 = summary_backward$adjr2)

metrics_backward
```

```
##      model_size      aic      bic      adjR2
## 1           1 90.347904 -74.0415 0.04778049
## 2           2 38.777408 -118.4034 0.07433592
## 3           3 31.547511 -119.9966 0.07848930
## 4           4 24.397483 -121.5566 0.08260688
## 5           5 20.526168 -119.8963 0.08506852
## 6           6 14.251140 -120.6568 0.08875097
## 7           7  8.866232 -120.5615 0.09198623
## 8           8  9.131293 -114.8097 0.09235882
## 9           9  9.777926 -108.6759 0.09253815
## 10          10 11.000000 -101.9640 0.09242543
```

```
# Finding which model has the lowest Cp and BIC
c("Lowest Cp" = which.min(summary_backward$cp),
  "Lowest BIC" = which.min(summary_backward$bic),
  "Highest Adj R2" = which.max(summary_backward$adjr2))
```

```
##      Lowest Cp      Lowest BIC Highest Adj R2
##              7              4              9
```

We will use a model of size 7 because it has a higher R^2 than the model of size 4.

```
set.seed(123)

# Fitting the model
backward_final <- regsubsets(Percent_Bleaching ~ .,
                             data = test,
                             nvmax = 10,
                             method = "backward")

# Model summary
summary_backward_final <- summary(backward_final)

# Reporting coefficients of best model
coef(backward_final, 7)
```

```
##      (Intercept) Distance_to_Shore      Turbidity Cyclone_Frequency
##      -5.825333e+02      2.664414e-04      -2.898639e+01      -8.067351e-02
##      Depth_m Temperature_Kelvin      SSTA      SSTA_DHW
##      -1.728421e-01      1.997514e+00      -2.608972e+00      5.374973e-01
```

Model 1.3: Ridge Regression

Below we will fit a ridge regression model with λ chosen by cross-validation.

```
##### Model 1.3 #####
# Model 1.3: Ridge Regression

set.seed(123)

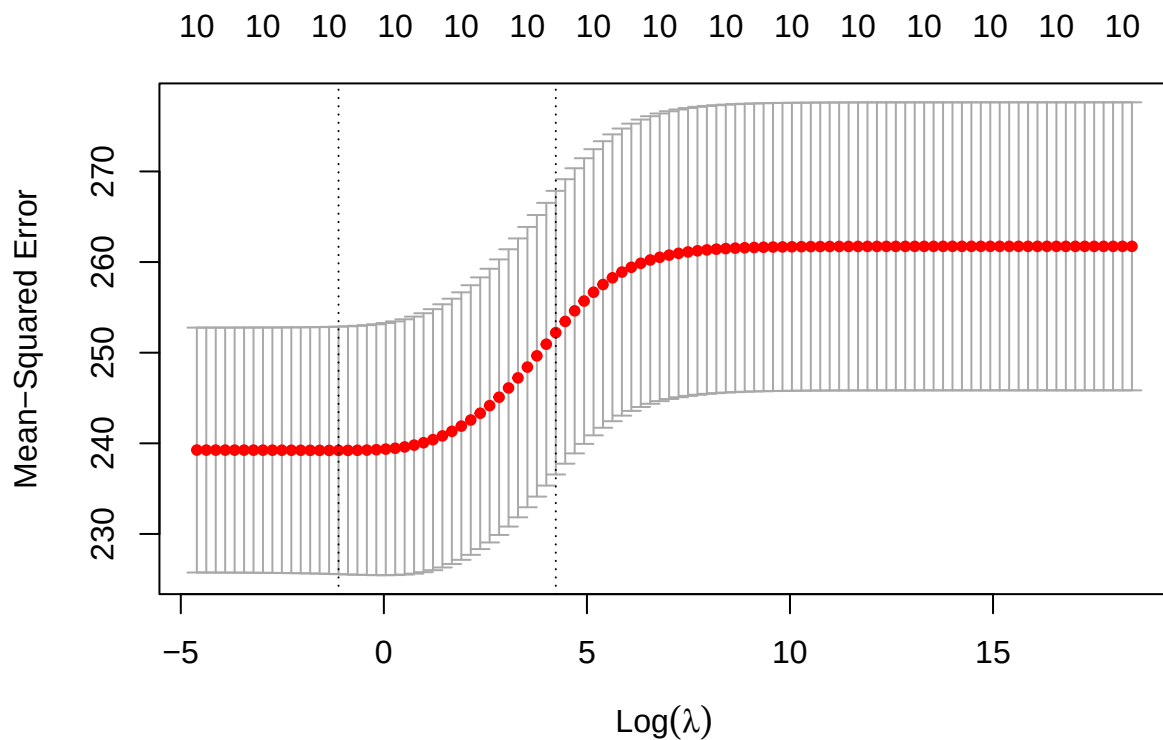
X_train <- model.matrix(Percent_Bleaching ~ ., data = train)[, -1]
X_test  <- model.matrix(Percent_Bleaching ~ ., data = test)[, -1]

Y_train <- train$Percent_Bleaching
Y_test  <- test$Percent_Bleaching

# Grid of values for lambda
grid <- 10^seq(-2, 8, length = 100)

# Fitting the model
ridge_fit <- cv.glmnet(x = X_train,
                      y = Y_train,
                      alpha = 0,
                      lambda = grid)

plot(ridge_fit)
```



```
# Optimal lambda value
lambda_min <- ridge_fit$lambda.min
lambda_1se <- ridge_fit$lambda.1se
```

```

# Prediction and finding test MSE
ridge_pred <- predict(ridge_fit, s = lambda_1se, newx = X_test)
ridge_error <- mean( (test$Percent_Bleaching - ridge_pred)^2 )
paste("Our estimated test MSE is", round(ridge_error, 2))

```

```
## [1] "Our estimated test MSE is 228.15"
```

```

# Compute test RMSE
ridge_rmse <- sqrt(ridge_error)
paste("Our estimated test RMSE is", round(ridge_rmse, 2))

```

```
## [1] "Our estimated test RMSE is 15.1"
```

```

# Reporting the coefficients
predict(ridge_fit, s = lambda_1se, type = "coefficients")[1:11, ]

```

```

##      (Intercept) Distance_to_Shore      Turbidity Cyclone_Frequency
##      -80.802036455      0.000101622      -3.555583013      -0.020139633
##      Depth_m      Windspeed Temperature_Kelvin      ClimSST
##      -0.022123817      -0.058994825      0.316161583      0.005166532
##      SSTA      SSTA_DHW ExposureSheltered
##      0.391399753      0.068859553      -0.219649421

```

Model 1.4: LASSO Regression

Below we will fit a LASSO regression model with λ chosen by cross-validation.

```

##### Model 1.4 #####
# Model 1.4: LASSO Regression

set.seed(123)

X_train <- model.matrix(Percent_Bleaching ~ ., data = train)[, -1]
X_test  <- model.matrix(Percent_Bleaching ~ ., data = test)[, -1]

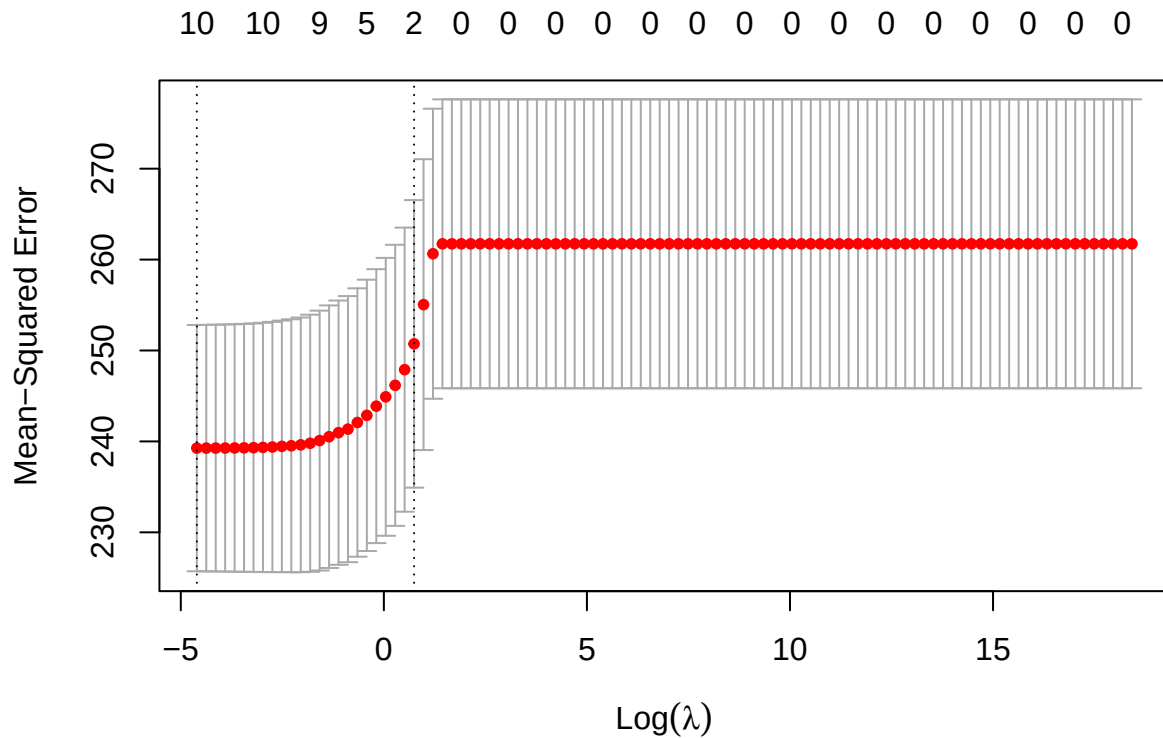
Y_train <- train$Percent_Bleaching
Y_test  <- test$Percent_Bleaching

# Grid of values for lambda
grid <- 10^seq(-2, 8, length = 100)

# Fitting the model
lasso_fit <- cv.glmnet(x = X_train,
                      y = Y_train,
                      alpha = 1,
                      lambda = grid)

plot(lasso_fit)

```



```
# Optimal lambda value
lambda_min <- lasso_fit$lambda.min
lambda_1se <- lasso_fit$lambda.1se

# Prediction and finding test MSE
lasso_pred <- predict(lasso_fit, s = lambda_1se, newx = X_test)
lasso_error <- mean( (test$Percent_Bleaching - lasso_pred)^2 )
paste("Our estimated test MSE is", round(lasso_error, 2))
```

```
## [1] "Our estimated test MSE is 226.19"
```

```
# Compute test RMSE
lasso_rmse <- sqrt(lasso_error)
paste("Our estimated test RMSE is", round(lasso_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 15.04"
```

```
# Reporting the coefficients
predict(lasso_fit, s = lambda_1se, type = "coefficients")[1:11, ]
```

```
##      (Intercept) Distance_to_Shore      Turbidity Cyclone_Frequency
##      -196.1987271      0.0000000      0.0000000      0.0000000
##      Depth_m      Windspeed Temperature_Kelvin      ClimSST
##      0.0000000      0.0000000      0.6981511      0.0000000
```

```
##          SSTA          SSTA_DHW ExposureSheltered
##          0.0000000          0.1110817          0.0000000
```

The only non-zero coefficients are for the following variables:

- Temperature_Kelvin
- SSTA_DHW

Model 1.5: Principal Components Regression (PCR)

Below we will fit a principal components regression model with M chosen by cross-validation.

```
##### Model 1.5 #####
# Model 1.5: Principal Components Regression (PCR)

set.seed(123)

# Fitting the model
pcr_fit <- pcr(Percent_Bleaching ~ .,
               data = train,
               center = TRUE,
               scale=TRUE,
               validation = "CV")
summary(pcr_fit)
```

```
## Data:      X dimension: 1798 10
## Y dimension: 1798 1
## Fit method: svdpc
## Number of components considered: 10
##
## VALIDATION: RMSEP
## Cross-validated using 10 random segments.
##      (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps  6 comps
## CV           16.18   16.02   15.72   15.72   15.72   15.71   15.68
## adjCV        16.18   16.03   15.72   15.72   15.72   15.71   15.68
##      7 comps  8 comps  9 comps 10 comps
## CV           15.68   15.66   15.61   15.45
## adjCV        15.67   15.66   15.60   15.44
##
## TRAINING: % variance explained
##           1 comps  2 comps  3 comps  4 comps  5 comps  6 comps
## X           22.031  43.43   57.454  68.705  78.40   84.979
## Percent_Bleaching  1.957   5.70   5.772   6.034   6.16   6.729
##           7 comps  8 comps  9 comps 10 comps
## X           90.218  94.790  97.785 100.000
## Percent_Bleaching  6.867   7.123   7.851   9.748
```

We can see above (and as is the case for PCR) we do not have 100% of our variance explained until we get to all $M = p = 10$ components, which is also where the lowest cross-validation error occurs. To combat including all 10 components, we can say that we would like to have at least 85% of our variance explained by our model, and therefore we would chose a model with $M = 7$ components!

```
# Prediction and finding test MSE (using 7 components)
pcr_pred <- predict(pcr_fit, test, ncomp = 7)
pcr_error <- mean( (test$Percent_Bleaching - pcr_pred)**2 )
paste("Our estimated test MSE is", round(pcr_error, 2))
```

```
## [1] "Our estimated test MSE is 225.13"
```

```
# Compute test RMSE
pcr_rmse <- sqrt(pcr_error)
paste("Our estimated test RMSE is", round(pcr_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 15"
```

Model 1.6: Partial Least Squares Regression (PLS)

Below we will fit a partial least squares regression model with M chosen by cross-validation.

```
##### Model 1.6 #####
# Model 1.6: Partial Least Squares Regression (PLS)
```

```
set.seed(123)
```

```
# Fitting the model
pls_fit <- plsr(Percent_Bleaching ~ .,
               data = train,
               center = TRUE,
               scale=TRUE,
               validation = "CV")
summary(pls_fit)
```

```
## Data:      X dimension: 1798 10
## Y dimension: 1798 1
## Fit method: kernelpls
## Number of components considered: 10
##
## VALIDATION: RMSEP
## Cross-validated using 10 random segments.
##      (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps  6 comps
## CV           16.18   15.63   15.52   15.47   15.46   15.45   15.45
## adjCV        16.18   15.63   15.52   15.47   15.45   15.45   15.44
##      7 comps  8 comps  9 comps 10 comps
## CV           15.45   15.45   15.45   15.45
## adjCV        15.44   15.44   15.44   15.44
##
## TRAINING: % variance explained
##      1 comps  2 comps  3 comps  4 comps  5 comps  6 comps
## X           21.110  28.369  38.65   53.92   64.298  74.575
## Percent_Bleaching  7.111   8.995   9.54   9.68   9.736   9.744
##      7 comps  8 comps  9 comps 10 comps
## X           81.353  89.007  94.969 100.000
## Percent_Bleaching  9.747   9.748   9.748   9.748
```

We can see above that once we get to $M = 5$ components, we reach the lowest cross-validation error. However, we do not have at least 85% of our variance explained until we get to $M = 8$ components. Therefore, we will choose a model with $M = 8$ components!

```
# Prediction and finding test MSE (using 8 components)
pls_pred <- predict(pls_fit, test, ncomp = 8)
pls_error <- mean( (test$Percent_Bleaching - pls_pred)**2 )
paste("Our estimated test MSE is", round(pls_error, 2))
```

```
## [1] "Our estimated test MSE is 216.76"
```

```
# Compute test RMSE
pls_rmse <- sqrt(pls_error)
paste("Our estimated test RMSE is", round(pls_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 14.72"
```

(Section 2) Nonparametric Regression Models

Model 2.1: K-Nearest Neighbors (KNN)

Below we will fit a KNN model with K chosen by cross-validation.

```
##### Model 2.1 #####
# Model 2.1: KNN Regression (KNN)

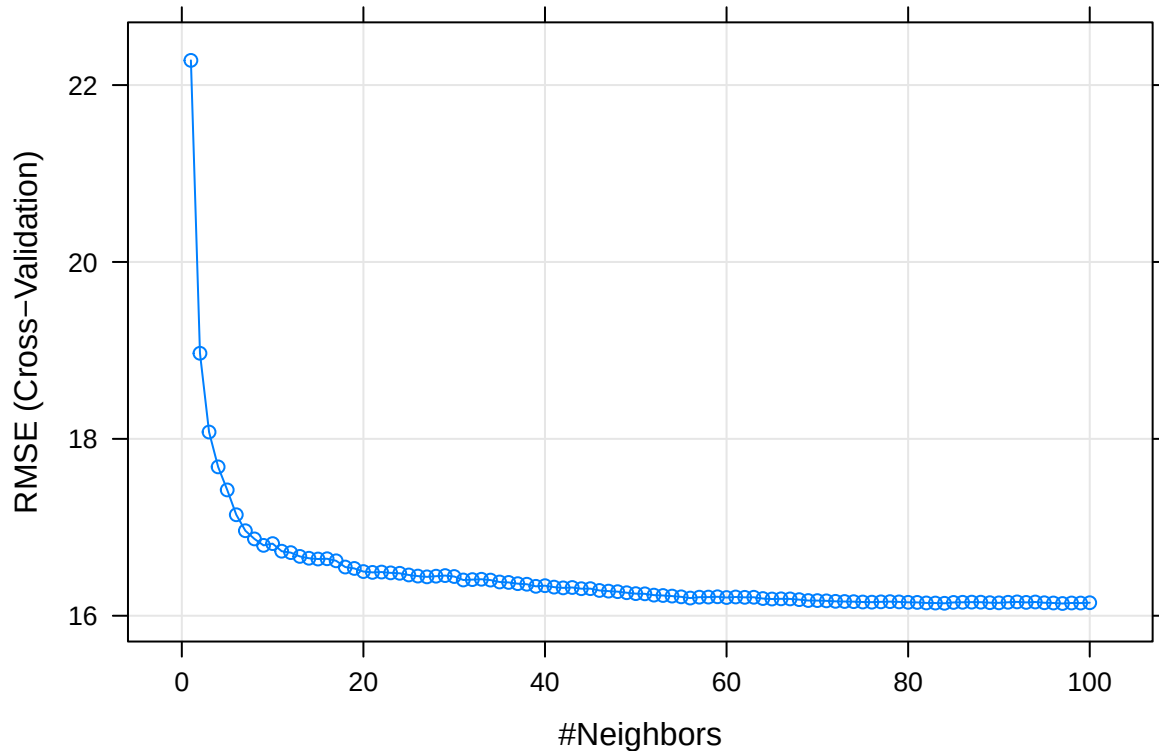
set.seed(123)

# Setting K grid
kgrid <- expand.grid(k = c(1:100))

# Training control parameters
cv <- trainControl(method = "cv",
                   number = 5)

# Fitting the model
knn_fit <- train(Percent_Bleaching ~ .,
                data = train,
                method = "knn",
                tuneGrid = kgrid,
                trControl = cv)

# Plotting knn_fit
plot(knn_fit)
```



```
# Finding optimal K
k_opt <- knn_fit$bestTune$k
paste("Our optimal K value is", k_opt)
```

```
## [1] "Our optimal K value is 97"
```

```
# Refitting with optimal K
knn_fit <- train(Percent_Bleaching ~ .,
  data = train,
  method = "knn",
  tuneGrid = expand.grid(k = k_opt),
  trControl = trainControl(method = "none"))
```

```
# Predict response for the test (holdout) set
knn_pred <- predict(knn_fit, newdata = test)
```

```
# Compute test MSE
knn_error <- mean((test$Percent_Bleaching - knn_pred)**2)
paste("Our estimated test MSE is", round(knn_error, 2))
```

```
## [1] "Our estimated test MSE is 237.23"
```

```
# Compute test RMSE
knn_rmse <- sqrt(knn_error)
paste("Our estimated test RMSE is", round(knn_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 15.4"
```

Model 2.2: Multivariate Adaptive Regression Splines (MARS)

Below we will fit a multivariate adaptive regression splines model.

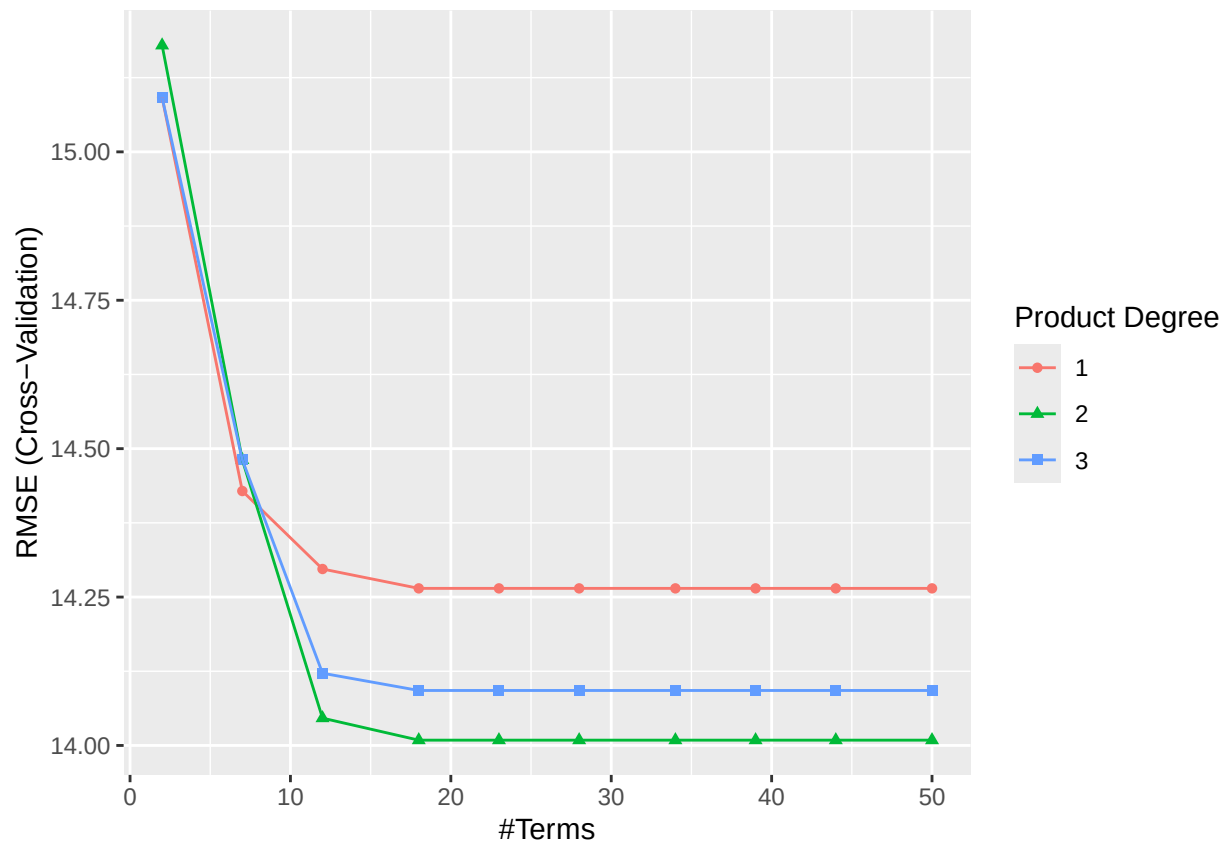
```
##### Model 2.2 #####
# Model 2.2: Multivariate Adaptive Regression Splines (MARS)

set.seed(123)

# Create tuning grid
grid <- expand.grid(degree = 1:3,
                   nprune = seq(2, 50, length.out = 10) %>%
                     floor())

# Fitting the model
mars_fit <- train(Percent_Bleaching ~ .,
                 data = train,
                 method = "earth",
                 metric = "RMSE",
                 trControl = trainControl(method = "cv", number = 10),
                 tuneGrid = grid)

ggplot(mars_fit)
```



```
# Displaying model with lowest training RMSE
mars_fit$results %>%
  filter(nprune == mars_fit$bestTune$nprune,
         degree == mars_fit$bestTune$degree)
```

```
##   degree nprune    RMSE Rsquared    MAE    RMSESD RsquaredSD    MAESD
## 1      2      18 14.00892 0.2544508 10.06711 0.8487127 0.06792531 0.4140104
```

```
# Predict response for the test (holdout) set
mars_pred <- predict(mars_fit, newdata = test)
```

```
# Compute test MSE
mars_error <- mean((test$Percent_Bleaching - mars_pred)**2)
paste("Our estimated test MSE is", round(mars_error, 2))
```

```
## [1] "Our estimated test MSE is 175.95"
```

```
# Compute test RMSE
mars_rmse <- sqrt(mars_error)
paste("Our estimated test RMSE is", round(mars_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 13.26"
```

Model 2.3: Generalized Additive Model (GAM)

Below we will fit a generalized additive model.

```
##### Model 2.3 #####
# Model 2.3: Generalized Additive Model (GAM)

set.seed(123)

require(gam)
require(splines)

# Fitting the model
gam_fit <- gam(Percent_Bleaching ~ Exposure
              + s(Distance_to_Shore)
              + s(Turbidity)
              + s(Cyclone_Frequency)
              + s(Depth_m)
              + s(Windspeed)
              + s(Temperature_Kelvin)
              + s(ClimSST)
              + s(SSTA)
              + s(SSTA_DHW),
              data = train)
summary(gam_fit)

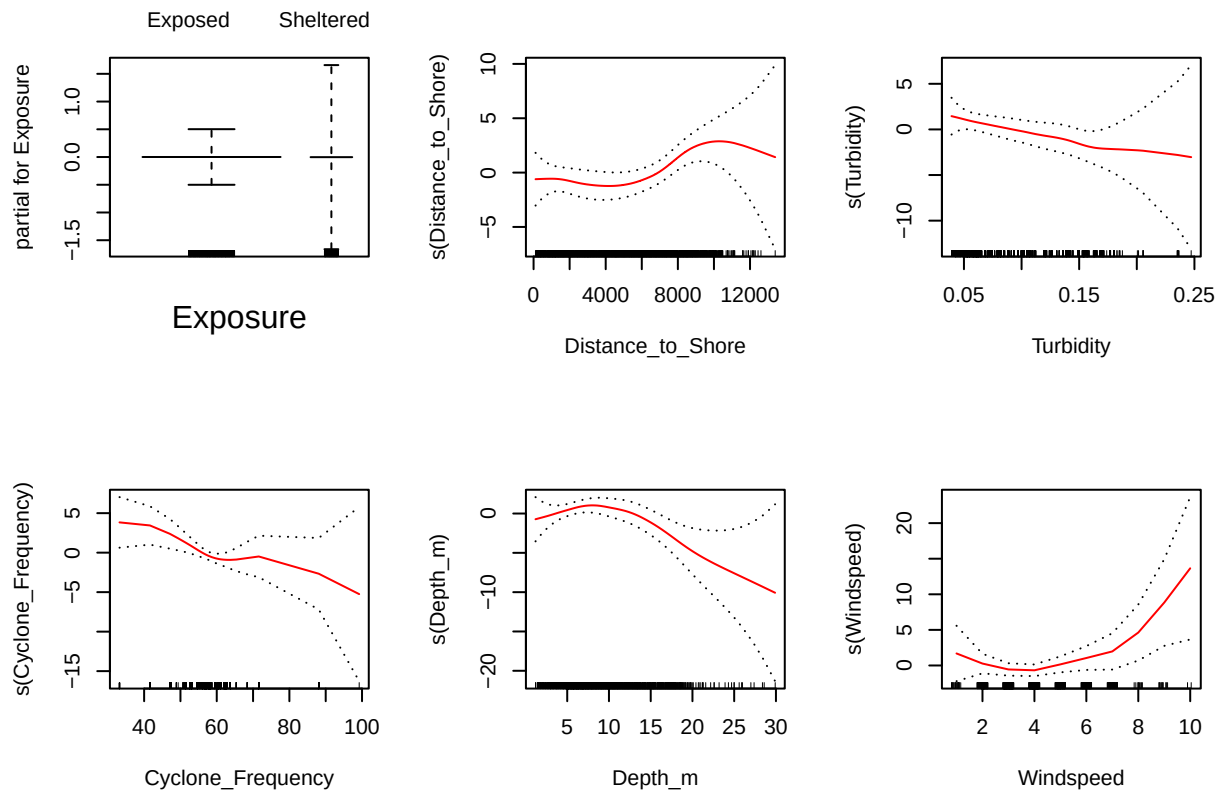
##
## Call: gam(formula = Percent_Bleaching ~ Exposure + s(Distance_to_Shore) +
##      s(Turbidity) + s(Cyclone_Frequency) + s(Depth_m) + s(Windspeed) +
##      s(Temperature_Kelvin) + s(ClimSST) + s(SSTA) + s(SSTA_DHW),
##      data = train)
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -29.096  -8.908  -2.833   5.193  74.445
##
## (Dispersion Parameter for gaussian family taken to be 198.033)
##
##      Null Deviance: 470411.8 on 1797 degrees of freedom
## Residual Deviance: 348538 on 1760 degrees of freedom
## AIC: 14650.7
##
## Number of Local Scoring Iterations: NA
##
## Anova for Parametric Effects
##

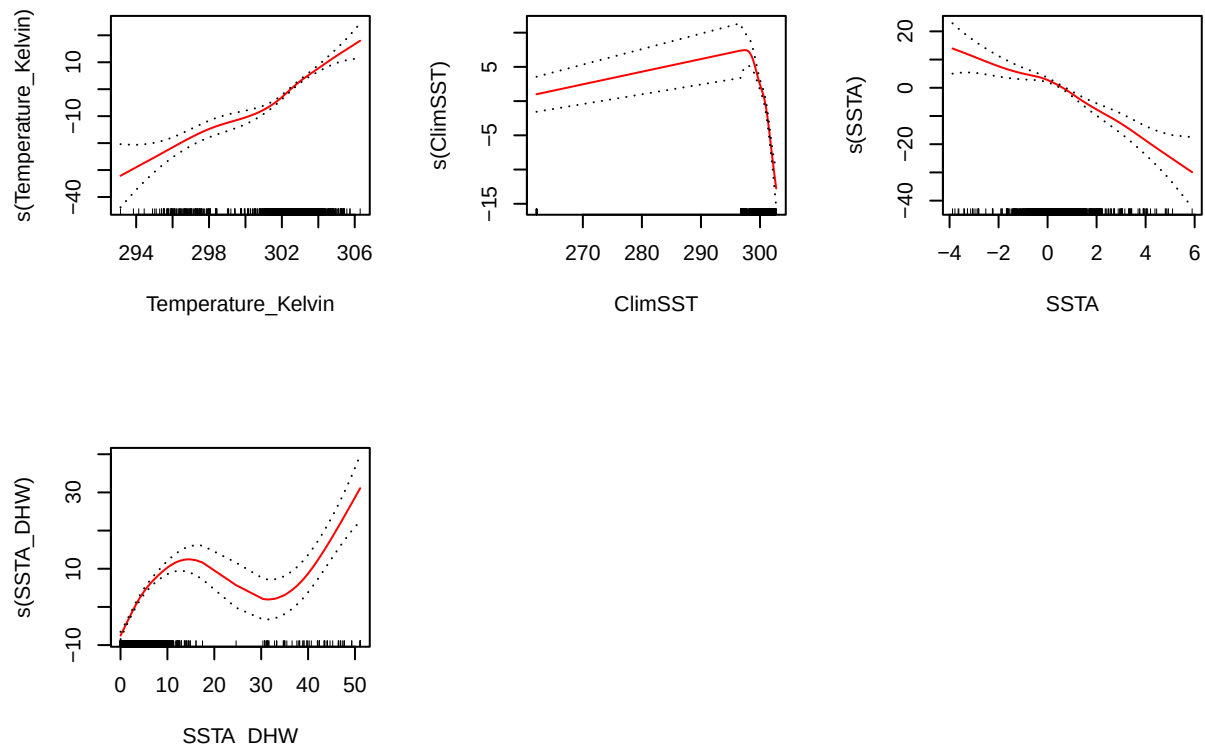

|                          | Df | Sum Sq | Mean Sq | F value  | Pr(>F)    |     |
|--------------------------|----|--------|---------|----------|-----------|-----|
| ## Exposure              | 1  | 823    | 823     | 4.1567   | 0.041618  | *   |
| ## s(Distance_to_Shore)  | 1  | 6512   | 6512    | 32.8845  | 1.149e-08 | *** |
| ## s(Turbidity)          | 1  | 216    | 216     | 1.0892   | 0.296790  |     |
| ## s(Cyclone_Frequency)  | 1  | 1104   | 1104    | 5.5740   | 0.018337  | *   |
| ## s(Depth_m)            | 1  | 1860   | 1860    | 9.3947   | 0.002209  | **  |
| ## s(Windspeed)          | 1  | 5196   | 5196    | 26.2368  | 3.352e-07 | *** |
| ## s(Temperature_Kelvin) | 1  | 38611  | 38611   | 194.9748 | < 2.2e-16 | *** |
| ## s(ClimSST)            | 1  | 1251   | 1251    | 6.3190   | 0.012034  | *   |
| ## s(SSTA)               | 1  | 771    | 771     | 3.8919   | 0.048677  | *   |


```

```
## s(SSTA_DHW)          1  21214   21214 107.1235 < 2.2e-16 ***
## Residuals            1760 348538    198
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Anova for Nonparametric Effects
##              Npar Df Npar F      Pr(F)
## (Intercept)
## Exposure
## s(Distance_to_Shore)      3  2.535  0.055291 .
## s(Turbidity)              3  0.201  0.895721
## s(Cyclone_Frequency)      3  1.293  0.275274
## s(Depth_m)                3  3.295  0.019798 *
## s(Windspeed)              3  3.860  0.009108 **
## s(Temperature_Kelvin)     3  8.084 2.388e-05 ***
## s(ClimSST)                3 42.825 < 2.2e-16 ***
## s(SSTA)                  3  3.852  0.009220 **
## s(SSTA_DHW)              3 70.183 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
par(mfrow = c(2,3))
plot(gam_fit, se = TRUE, col = "red")
```





We have very strong nonlinear relationships between Percent Bleaching and both ClimSST and SSTA_DHW. We have a moderately strong nonlinear relationship between Percent Bleaching and Temperature_Kelvin.

```
# Predict response for the test (holdout) set
gam_pred <- predict(gam_fit, newdata = test)

# Compute test MSE
gam_error <- mean((test$Percent_Bleaching - gam_pred)**2)
paste("Our estimated test MSE is", round(gam_error, 2))
```

```
## [1] "Our estimated test MSE is 182.47"
```

```
# Compute test RMSE
gam_rmse <- sqrt(gam_error)
paste("Our estimated test RMSE is", round(gam_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 13.51"
```

Model 2.4: Local Regression (LR)

Below we will fit a local regression model.

```
##### Model 2.4 #####
# Model 2.4: Local Regression (LR)
```

```

set.seed(123)

# Training control parameters and tuning grid
cv <- trainControl(method = "cv", number = 10)
grid <- expand.grid(span = seq(0.5, 0.9, len = 5), degree = 1)

# Fitting the model
local_model <- train(Percent_Bleaching ~ .,
                     data = train,
                     method = "gamLoess",
                     trControl = cv,
                     tuneGrid = grid)

local_model

```

```

## Generalized Additive Model using LOESS
##
## 1798 samples
## 10 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1618, 1617, 1619, 1619, 1618, 1619, ...
## Resampling results across tuning parameters:
##
##  span  RMSE      Rsquared  MAE
##  0.5   14.18750  0.2330269  10.37584
##  0.6   14.23107  0.2280273  10.43072
##  0.7   14.39777  0.2095257  10.57709
##  0.8   14.41902  0.2062051  10.57635
##  0.9   14.48882  0.1976515  10.59419
##
## Tuning parameter 'degree' was held constant at a value of 1
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were span = 0.5 and degree = 1.

```

We can see above that the optimal value for span is 0.5.

```

set.seed(123)

# Fitting the model
local_model2 <- train(Percent_Bleaching ~ .,
                     data = train,
                     method = "gamLoess",
                     trControl = trainControl(method = "none"),
                     tuneGrid = expand.grid(span = 0.5, degree = 1))

# Predict response for the test (holdout) set
local_pred <- predict(local_model2, newdata = test)

# Compute test MSE
local_error <- mean((test$Percent_Bleaching - local_pred)**2)
paste("Our estimated test MSE is", round(local_error, 2))

```

```
## [1] "Our estimated test MSE is 182.85"
```

```
# Compute test RMSE
local_rmse <- sqrt(local_error)
paste("Our estimated test RMSE is", round(local_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 13.52"
```

Model 2.5: Support Vector Regression (SVR)

Below we will fit a support vector regression model.

```
##### Model 2.5 #####
# Model 2.5: Support Vector Machine (SVM)
```

```
set.seed(123)

svr_fit <- svm(Percent_Bleaching ~ .,
              data = train,
              type = "eps-regression",
              kernel = "radial",
              gamma = 1,
              cost = 1,
              epsilon = 0.1)
summary(svr_fit)
```

```
##
## Call:
## svm(formula = Percent_Bleaching ~ ., data = train, type = "eps-regression",
##      kernel = "radial", gamma = 1, cost = 1, epsilon = 0.1)
##
##
## Parameters:
##   SVM-Type:  eps-regression
##   SVM-Kernel: radial
##      cost:   1
##     gamma:   1
##    epsilon: 0.1
##
##
## Number of Support Vectors: 1547
```

```
# Predict response for the test (holdout) set
svr_pred <- predict(svr_fit, newdata = test)

# Compute test MSE
svr_error <- mean((test$Percent_Bleaching - svr_pred)**2)
paste("Our estimated test MSE is", round(svr_error, 2))
```

```
## [1] "Our estimated test MSE is 182.42"
```

```
# Compute test RMSE
svr_rmse <- sqrt(svr_error)
paste("Our estimated test RMSE is", round(svr_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 13.51"
```

(Section 3) Regression Tree Models

Model 3.1: Standard Regression Tree

Below we will fit a standard regression tree model.

```
##### Model 3.1 #####
# Model 3.1: Standard Regression Tree
```

```
set.seed(123)

# Building the initial tree
tree_fit <- rpart(Percent_Bleaching ~ .,
                  data = train,
                  control = rpart.control(cp=.0001))
#printcp(tree_fit)

# Identify the best cp value to use
best_cp <- tree_fit$cptable[which.min(tree_fit$cptable[, "xerror"]), "CP"]

# Pruned tree based on the best cp value
pruned_tree <- prune(tree_fit, cp = best_cp)

jpeg(file="regressiontree.jpeg")

# Plot the pruned tree
tree_plot <- prp(pruned_tree,
                 faclen = 0, # Use full names for factor labels
                 extra = 1, # Display number of obs for each terminal node
                 roundint = F, # Don't round to integers in output
                 digits = 5) # Display 5 decimal places in output
#dev.off()
```

```
# Predict response for the test (holdout) set
tree_pred <- predict(tree_fit, newdata = test)

# Compute test MSE
tree_error <- mean((test$Percent_Bleaching - tree_pred)**2)
paste("Our estimated test MSE is", round(tree_error, 2))
```

```
## [1] "Our estimated test MSE is 151.27"
```

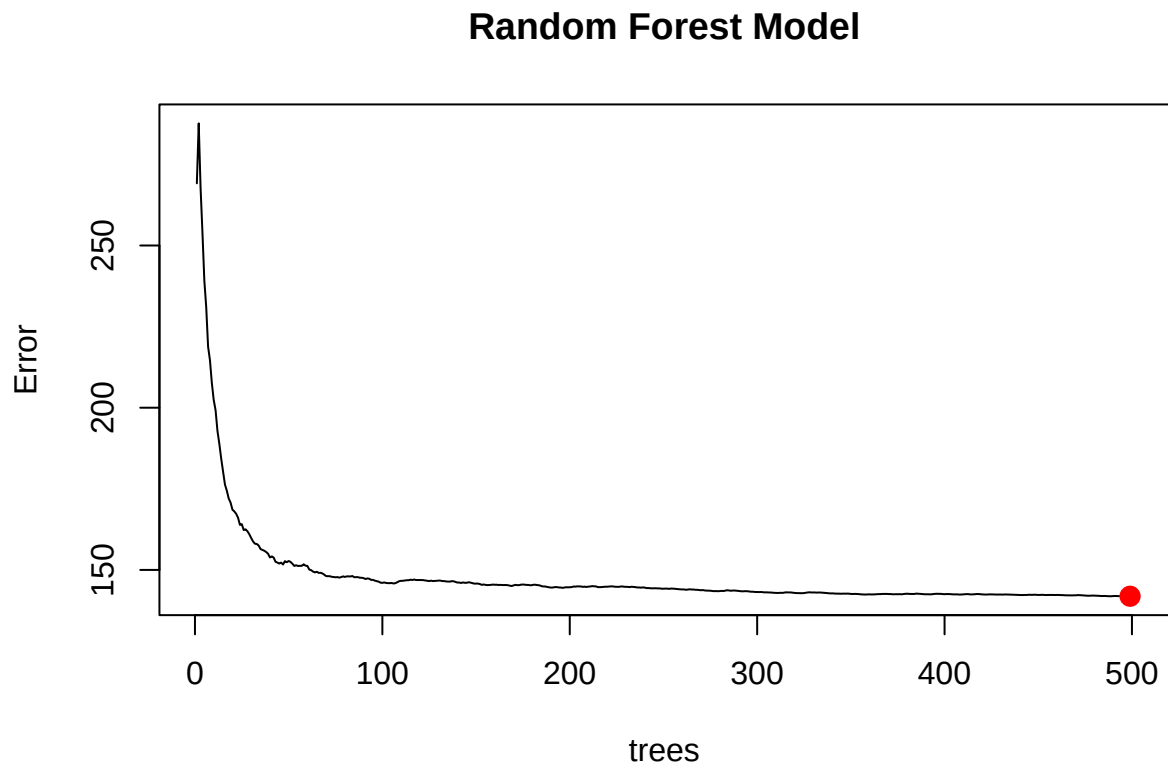
```
# Compute test RMSE
tree_rmse <- sqrt(tree_error)
paste("Our estimated test RMSE is", round(tree_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 12.3"
```

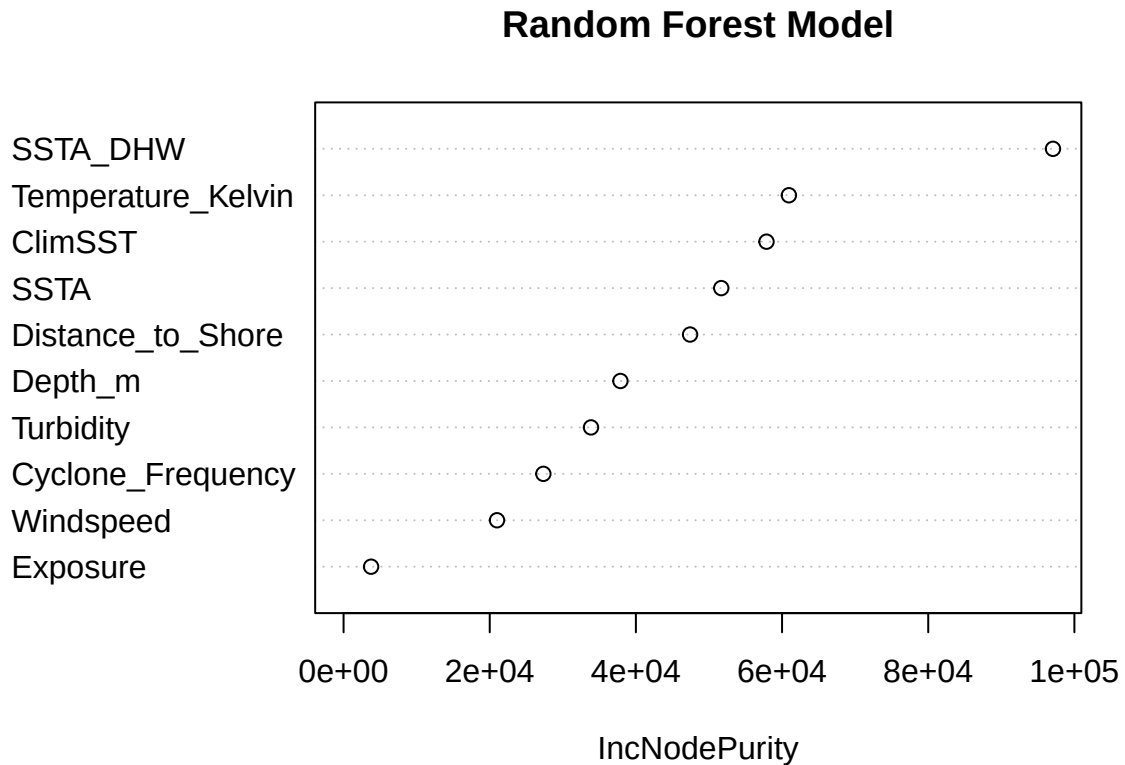
Model 3.2: Random Forest

Below we will fit a random forest model.

```
##### Model 3.2 #####  
# Model 3.2: Random Forest  
  
set.seed(123)  
  
# Fitting the model  
rf_fit <- randomForest(formula = Percent_Bleaching ~ .,  
                        data = train)  
  
# Number of trees that produced lowest test MSE  
which.min(rf_fit$mse)  
  
## [1] 499  
  
# Plotting the error vs number of trees  
plot(rf_fit, main = "Random Forest Model")  
points(which.min(rf_fit$mse), rf_fit$mse[which.min(rf_fit$mse)],  
       col = "red", cex = 2, pch = 20)
```



```
# Variable importance plot
varImpPlot(rf_fit, main = "Random Forest Model")
```

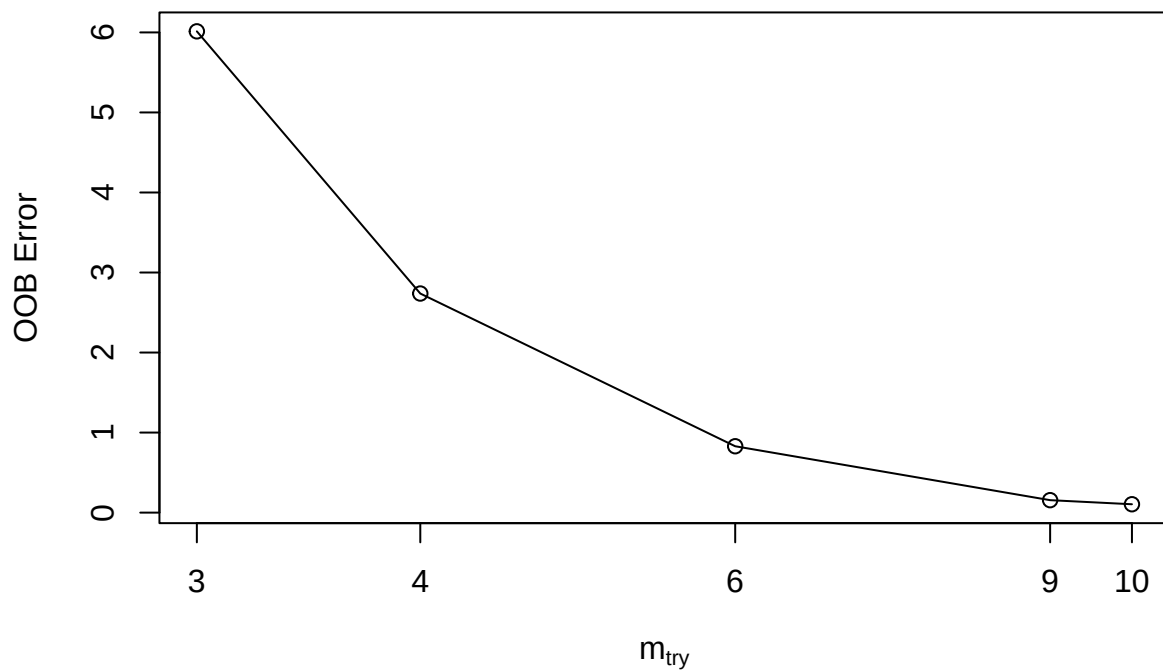


Below we will tune the model.

```
set.seed(123)

rf_tuned <- tuneRF(x = train[,-1],
                   y = train$Percent_Bleaching,
                   ntreeTry = 500,
                   mtryStart = 4,
                   stepFactor = 1.5,
                   improve = 0.01)
```

```
## mtry = 4  OOB error = 2.737314
## Searching left ...
## mtry = 3    OOB error = 6.013212
## -1.196757 0.01
## Searching right ...
## mtry = 6    OOB error = 0.8292
## 0.6970753 0.01
## mtry = 9    OOB error = 0.1556525
## 0.812286 0.01
## mtry = 10   OOB error = 0.1046437
## 0.3277094 0.01
```



```
rf_tuned
```

```
##      mtry  OOBError
## 3        3 6.0132123
## 4        4 2.7373138
## 6        6 0.8292000
## 9        9 0.1556525
## 10       10 0.1046437
```

The lowest OOB error is achieved by using 10 randomly selected predictors at each split when building the trees.

```
# Predict response for the test (holdout) set
rf_pred <- predict(rf_fit, newdata = test)

# Compute test MSE
rf_error <- mean((test$Percent_Bleaching - rf_pred)**2)
paste("Our estimated test MSE is", round(rf_error, 2))
```

```
## [1] "Our estimated test MSE is 121.88"
```

```
# Compute test RMSE
rf_rmse <- sqrt(rf_error)
paste("Our estimated test RMSE is", round(rf_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 11.04"
```

Model 3.3: Bagging

Below we will fit a bagged tree model.

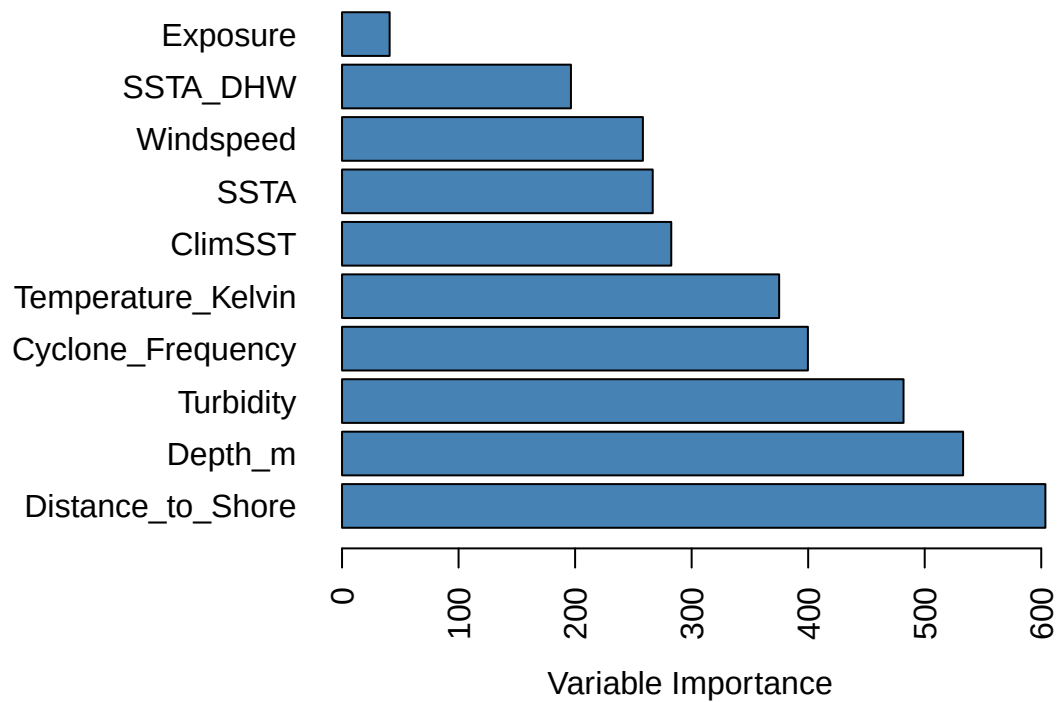
```
##### Model 3.3 #####  
# Model 3.3: Bagging  
  
set.seed(123)  
  
bag_fit <- bagging(formula = Percent_Bleaching ~ .,  
                  data = train,  
                  nbagg = 150,  
                  coob = TRUE,  
                  control = rpart.control(minsplit = 2, cp = 0))  
  
bag_fit  
  
##  
## Bagging regression trees with 150 bootstrap replications  
##  
## Call: bagging.data.frame(formula = Percent_Bleaching ~ ., data = train,  
##      nbagg = 150, coob = TRUE, control = rpart.control(minsplit = 2,  
##      cp = 0))  
##  
## Out-of-bag estimate of root mean squared error: 11.8836
```

Meaning of certain arguments:

- **nbagg**: Number of bootstrap samples.
- **coob**: Set to TRUE in order to obtain the estimated OOB error.
- **minsplit**: This tells the model to only require 2 observations in a node to split.
- **cp**: This is the complexity parameter. By setting it to 0, we don't require the model to be able to improve the overall fit by any amount in order to perform a split.

Essentially, these last two arguments allow the individual trees to grow extremely deep, which leads to trees with high variance, but low bias. Then when we apply bagging we're able to reduce the variance of the final model while keeping the bias low.

```
set.seed(123)  
  
# Calculate variable importance  
VI <- data.frame(var = names(coral_data[,-1]), imp = varImp(bag_fit))  
  
# Sort variable importance descending  
VI_plot <- VI[order(VI$Overall, decreasing=TRUE),]  
  
par(mar=c(4,10,4,4))  
  
# Visualize variable importance with horizontal bar plot  
barplot(VI_plot$Overall,  
        names.arg = rownames(VI_plot),  
        horiz = TRUE,  
        col = "steelblue",  
        xlab = "Variable Importance",  
        las = 2)
```



```
# Predict response for the test (holdout) set
bag_pred <- predict(bag_fit, newdata = test)

# Compute test MSE
bag_error <- mean((test$Percent_Bleaching - bag_pred)**2)
paste("Our estimated test MSE is", round(bag_error, 2))
```

```
## [1] "Our estimated test MSE is 120.44"
```

```
# Compute test RMSE
bag_rmse <- sqrt(bag_error)
paste("Our estimated test RMSE is", round(bag_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 10.97"
```

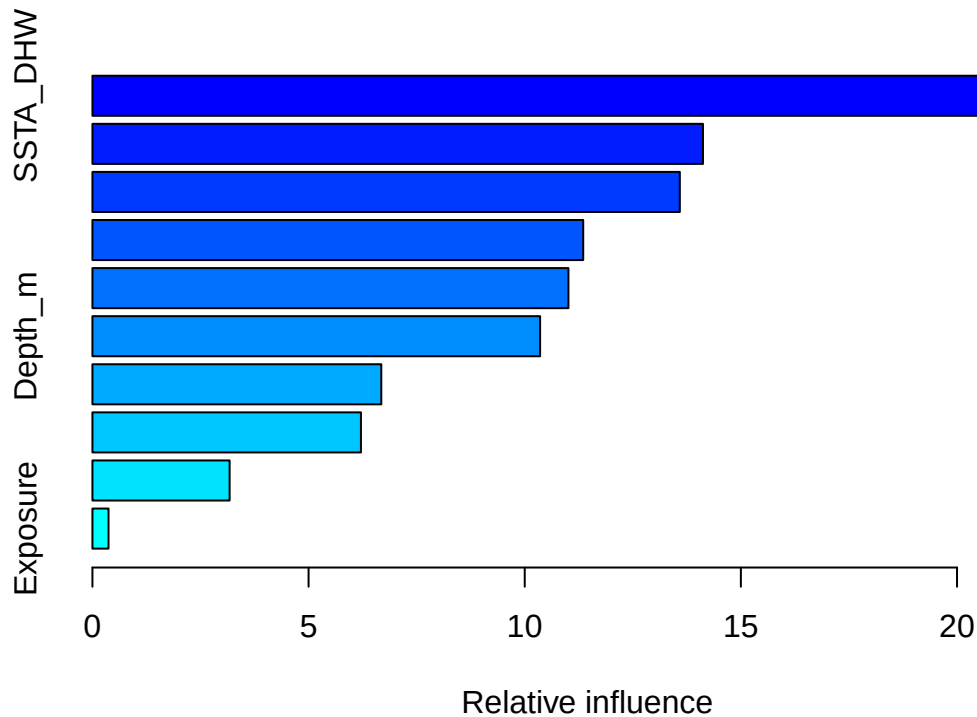
Model 3.4: Boosting

Below we will fit a boosted tree model.

```
##### Model 3.4 #####
# Model 3.4: Boosting

set.seed(123)
```

```
boost_fit <- gbm(Percent_Bleaching ~ .,
  data = train,
  distribution = "gaussian",
  n.trees = 5000,
  interaction.depth = 4)
summary(boost_fit)
```



```
##               var    rel.inf
## SSTA_DHW      SSTA_DHW 23.1307418
## Distance_to_Shore Distance_to_Shore 14.1229549
## ClimSST       ClimSST 13.5877954
## SSTA          SSTA 11.3529763
## Temperature_Kelvin Temperature_Kelvin 11.0110495
## Depth_m       Depth_m 10.3561369
## Turbidity     Turbidity 6.6817838
## Cyclone_Frequency Cyclone_Frequency 6.2114567
## Windspeed     Windspeed 3.1731208
## Exposure      Exposure 0.3719837
```

```
# Predict response for the test (holdout) set
boost_pred <- predict(boost_fit, newdata = test)

# Compute test MSE
boost_error <- mean((test$Percent_Bleaching - boost_pred)**2)
paste("Our estimated test MSE is", round(boost_error, 2))
```

```
## [1] "Our estimated test MSE is 155.51"
```

```
# Compute test RMSE
boost_rmse <- sqrt(boost_error)
paste("Our estimated test RMSE is", round(boost_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 12.47"
```

Model 3.5: Bayesian Additive Regression Tree (BART)

Below we will fit a Bayesian additive regression tree model.

```
##### Model 3.5 #####
# Model 3.5: Bayesian Additive Regression Tree (BART)
```

```
set.seed(123)

x <- train[,1:10]
y <- ("y" = train$Percent_Bleaching)

# Fitting the model
bart_fit <- bartMachine(X = x, y = y,
                        num_trees = 100)
```

```
## bartMachine initializing with 100 trees...
## bartMachine vars checked...
## bartMachine java init...
## bartMachine factors created...
## bartMachine before preprocess...
## bartMachine after preprocess... 11 total features...
## bartMachine sigsq estimated...
## bartMachine training data finalized...
## Now building bartMachine for regression...Covariate importance prior ON.
## evaluating in sample data...done
```

```
summary(bart_fit)
```

```
## bartMachine v1.3.4.1 for regression
##
## training data size: n = 1798 and p = 11
## built in 6.7 secs on 1 core, 100 trees, 250 burn-in and 1000 post. samples
##
## sigsq est for y beforehand: 236.259
## avg sigsq estimate after burn-in: 142.54051
##
## in-sample statistics:
## L1 = 14528.07
## L2 = 222692.61
## rmse = 11.13
## Pseudo-Rsq = 0.5266
## p-val for shapiro-wilk test of normality of residuals: 0
## p-val for zero-mean noise: 0.99124
```

```
# Predict response for the test (holdout) set
bart_pred <- bart_machine_get_posterior(bart_fit, new_data = test[,1:10])
```

```
# Compute test MSE
bart_error <- mean((test$Percent_Bleaching - bart_pred$y_hat)**2)
paste("Our estimated test MSE is", round(bart_error, 2))
```

```
## [1] "Our estimated test MSE is 143.22"
```

```
# Compute test RMSE
bart_rmse <- sqrt(bart_error)
paste("Our estimated test RMSE is", round(bart_rmse, 2))
```

```
## [1] "Our estimated test RMSE is 11.97"
```

(Section 4) Stacking Models

Model 4.1: GAM / Random Forest / GBM (Boosting)

Below we will fit a stacked model using GAM, Random Forest, and GBM (Boosting).

```
##### Model 4.1 #####
# Model 4.1: GAM / Random Forest / Boosted Tree
```

```
set.seed(123)
```

```
# Get response and feature names
Y <- "Percent_Bleaching"
X <- setdiff(names(train), Y)
```

```
# Init h2o
h2o.init()
```

```
## Connection successful!
##
## R is connected to the H2O cluster:
##   H2O cluster uptime:      16 hours 35 minutes
##   H2O cluster timezone:    America/New_York
##   H2O data parsing timezone: UTC
##   H2O cluster version:     3.46.0.6
##   H2O cluster version age:  1 month and 2 days
##   H2O cluster name:        H2O_started_from_R_Rachel_Hardy_osn291
##   H2O cluster total nodes:  1
##   H2O cluster total memory: 2.51 GB
##   H2O cluster total cores:  4
##   H2O cluster allowed cores: 4
##   H2O cluster healthy:     TRUE
##   H2O Connection ip:       localhost
##   H2O Connection port:     54321
##   H2O Connection proxy:    NA
##   H2O Internal Security:   FALSE
##   R Version:                R version 4.2.1 (2022-06-23 ucrt)
```

```
# Convert train/test sets to h2o format
coral_train <- as.h2o(train)
```

```
## |
```

```
coral_test <- as.h2o(test)
```

```
## |
```

```
nfolds <- 5
seed <- 123
```

```
# GAM
```

```
gam_model <- h2o.gam(x = X,
  y = Y,
  training_frame = coral_train,
  nfolds = nfolds,
  seed = seed,
  keep_cross_validation_predictions = TRUE,
  fold_assignment = "Modulo",
  family = "gaussian",
  gam_columns = c("Distance_to_Shore",
    "Turbidity",
    "Cyclone_Frequency",
    "Depth_m",
    "Temperature_Kelvin",
    "ClimSST",
    "SSTA"))
```

```
## |
```

```
# Random forest
```

```
rf_model <- h2o.randomForest(x = X,
  y = Y,
  training_frame = coral_train,
  nfolds = nfolds,
  seed = seed,
  keep_cross_validation_predictions = TRUE,
  fold_assignment = "Modulo",
  ntrees = 1000,
  mtries = 5,
  max_depth = 30,
  sample_rate = 0.8,
  stopping_rounds = 50,
  stopping_metric = "RMSE",
  stopping_tolerance = 0)
```

```
## |
```

```

# GBM (Boosting)
gbm_model <- h2o.gbm(x = X,
                     y = Y,
                     training_frame = coral_train,
                     nfolds = nfolds,
                     seed = seed,
                     keep_cross_validation_predictions = TRUE,
                     fold_assignment = "Modulo",
                     ntrees = 5000,
                     learn_rate = 0.01,
                     max_depth = 7,
                     min_rows = 5,
                     sample_rate = 0.8,
                     stopping_rounds = 50,
                     stopping_metric = "RMSE",
                     stopping_tolerance = 0)

```

```
## |
```

```

coral_ensemble <- h2o.stackedEnsemble(x = X,
                                      y = Y,
                                      training_frame = coral_train,
                                      model_id = "coral_ensemble",
                                      base_models = list(gam_model,
                                                         rf_model,
                                                         gbm_model),
                                      metalearner_algorithm = "AUTO")

```

```
## |
```

```

# Function to extract RMSE
get_rmse <- function(model){
  perf <- h2o.performance(model, newdata = coral_test)
  return(perf@metrics$RMSE)
}

# Function to extract MSE
get_mse <- function(model){
  perf <- h2o.performance(model, newdata = coral_test)
  return( (perf@metrics$RMSE)**2 )
}

# Model list
mod_list <- list(GAM = gam_model,
                 RF = rf_model,
                 GBM = gbm_model,
                 STACKED = coral_ensemble)

# Apply function to model list
ensemble_rmse <- purrr::map_dbl(mod_list, get_rmse)
ensemble_error <- purrr::map_dbl(mod_list, get_mse)

```

```

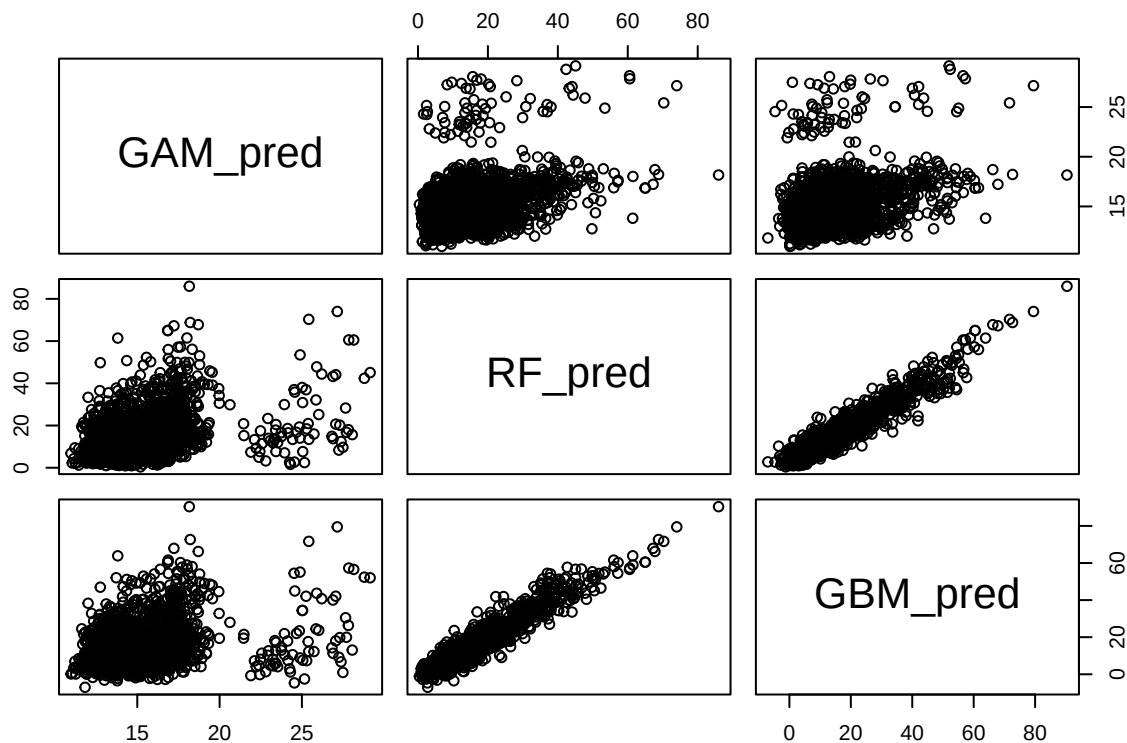
# Data frame of predictions
pred <- data.frame(
  GAM_pred = as.vector(h2o.getFrame(gam_model@model$cross_validation_holdout_predictions_frame_id$name)),
  RF_pred = as.vector(h2o.getFrame(rf_model@model$cross_validation_holdout_predictions_frame_id$name)),
  GBM_pred = as.vector(h2o.getFrame(gbm_model@model$cross_validation_holdout_predictions_frame_id$name))

# Correlation of the predictions
cor(pred)

##           GAM_pred  RF_pred  GBM_pred
## GAM_pred 1.0000000 0.3375500 0.2914079
## RF_pred   0.3375500 1.0000000 0.9491567
## GBM_pred   0.2914079 0.9491567 1.0000000

# Plotting the correlation of the predictions
plot(pred)

```



Summaries

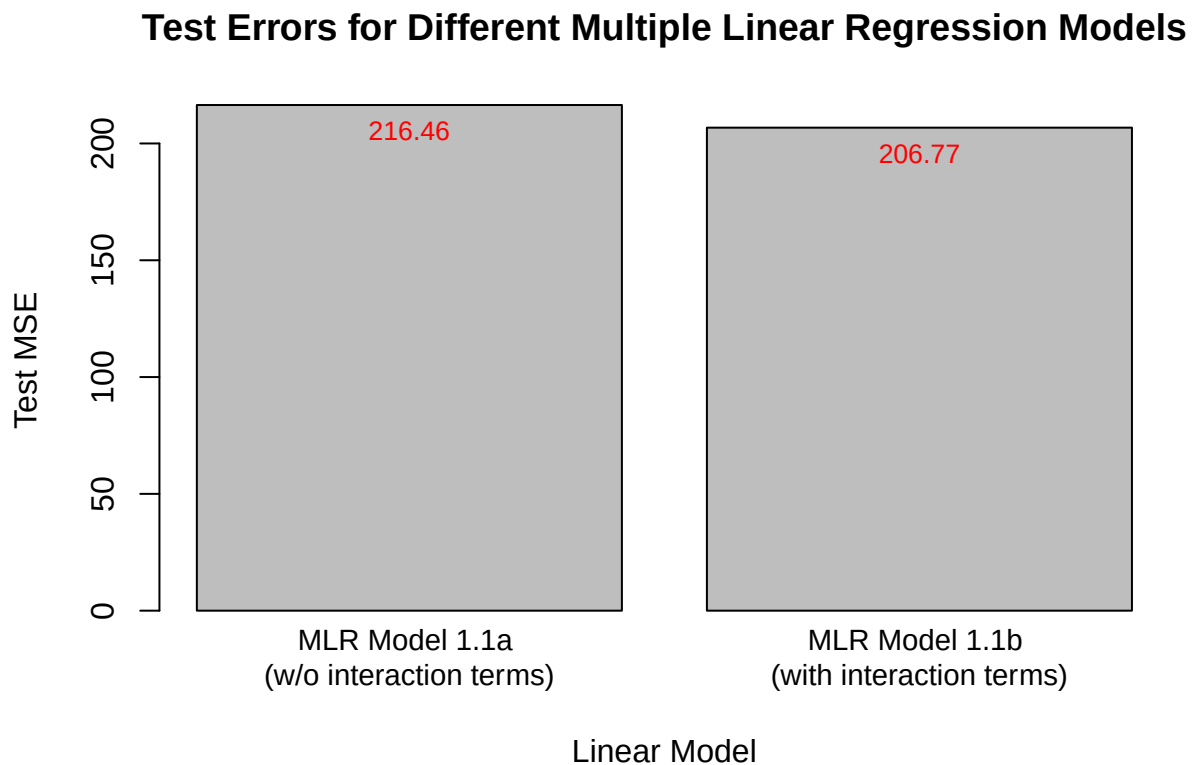
Linear Regression Models

```

# Combine the error values into a vector
lm_errors <- c(lm_error1, lm_error2)
names(lm_errors) <- c("MLR Model 1.1a \n(w/o interaction terms)",
                     "MLR Model 1.1b \n(with interaction terms)")

# Creating a bar plot of the errors
bar <- barplot(lm_errors,
               cex.names = 0.9,
               xlab = "Linear Model",
               ylab = "Test MSE",
               main = "Test Errors for Different Multiple Linear Regression Models")
text(x = bar, y = lm_errors, label = round(lm_errors,2),
     pos = 1, cex = 0.8, col = "red")

```



```

# Combine the error values into a vector
errors <- c(lm_error1, lm_error2, ridge_error,
            lasso_error, pcr_error, pls_error)
names(errors) <- c("MLR \n(1.1a)",
                  "MLR \n(1.1b)",
                  "Ridge \n(1.3)",
                  "Lasso \n(1.4)",
                  "PCR \n(1.5)",
                  "PLS \n(1.6)")

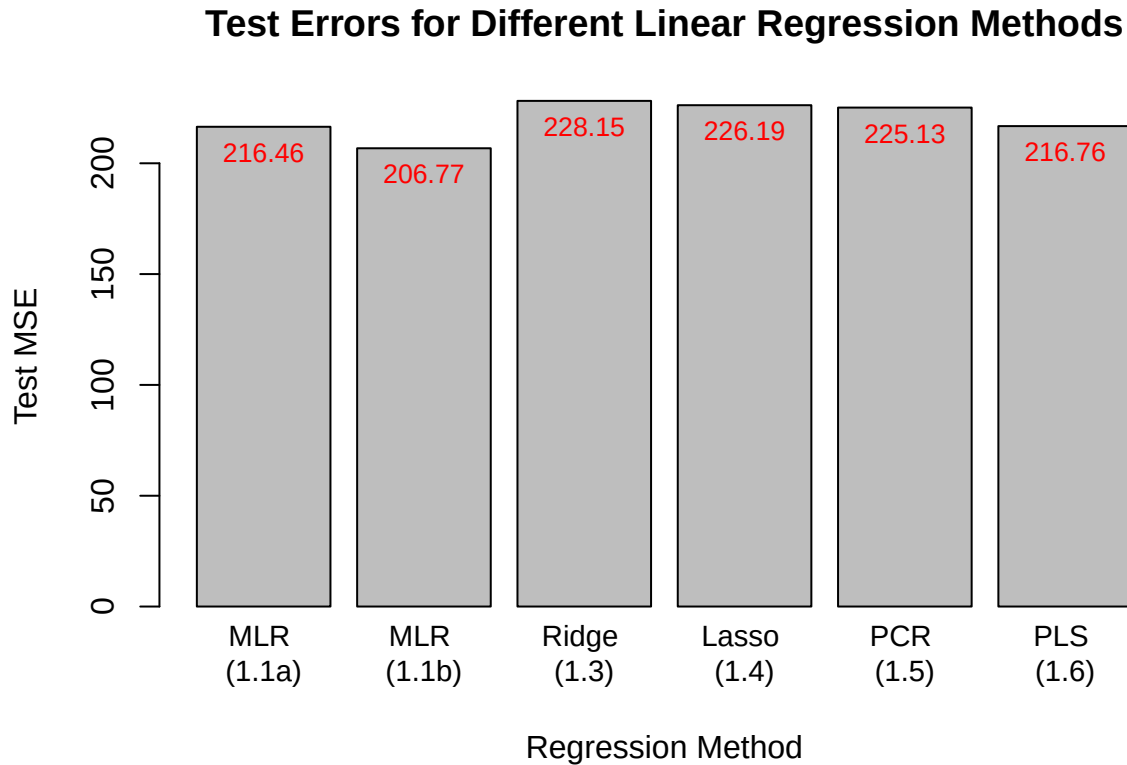
# Creating a bar plot of the errors

```

```

bar <- barplot(errors,
               cex.names = 0.9,
               xlab = "Regression Method",
               ylab = "Test MSE",
               main = "Test Errors for Different Linear Regression Methods")
text(x = bar, y = errors, label = round(errors,2),
     pos = 1, cex = 0.8, col = "red")

```



```

# MSE and RMSE values for each linear model
MSE <- c(lm_error1, lm_error2, ridge_error,
         lasso_error, pcr_error, pls_error)
RMSE <- c(lm_rmse1, lm_rmse2, ridge_rmse,
          lasso_rmse, pcr_rmse, pls_rmse)

# Combining the two vectors and adding row names
results <- data.frame("MSE" = MSE,
                     "RMSE" = RMSE)
rownames(results) <- c("MLR (w/o interaction)",
                     "MLR (with interaction)",
                     "Ridge", "LASSO",
                     "PCR", "PLS")

results

```

```

##              MSE      RMSE
## MLR (w/o interaction) 216.4616 14.71264

```

```
## MLR (with interaction) 206.7676 14.37942
## Ridge                 228.1491 15.10460
## LASSO                 226.1912 15.03966
## PCR                  225.1290 15.00430
## PLS                   216.7603 14.72278
```

```
kable(results)
```

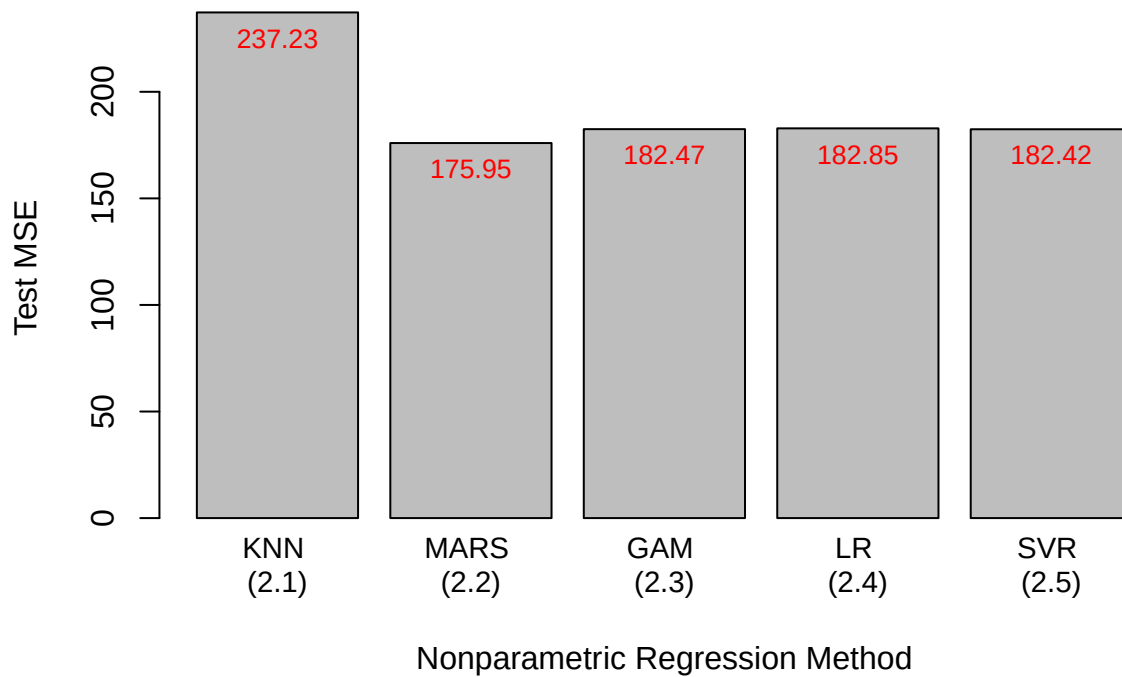
	MSE	RMSE
MLR (w/o interaction)	216.4616	14.71264
MLR (with interaction)	206.7676	14.37942
Ridge	228.1491	15.10460
LASSO	226.1912	15.03966
PCR	225.1290	15.00430
PLS	216.7603	14.72278

Nonparametric Regression Models

```
# Combine the error values into a vector
errors <- c(knn_error, mars_error,
            gam_error, local_error,
            svr_error)
names(errors) <- c("KNN \n(2.1)",
                  "MARS \n(2.2)",
                  "GAM \n(2.3)",
                  "LR \n(2.4)",
                  "SVR \n(2.5)")

# Creating a bar plot of the errors
bar <- barplot(errors,
               cex.names = 0.9,
               xlab = "Nonparametric Regression Method",
               ylab = "Test MSE",
               main = "Test Errors for Different Nonparametric Regression Methods")
text(x = bar, y = errors, label = round(errors,2),
     pos = 1, cex = 0.8, col = "red")
```

Test Errors for Different Nonparametric Regression Methods



```
# MSE and RMSE values for each nonparametric model
```

```
MSE <- c(knn_error, mars_error,  
         gam_error, local_error,  
         svr_error)
```

```
RMSE <- c(knn_rmse, mars_rmse,  
          gam_rmse, local_rmse,  
          svr_rmse)
```

```
# Combining the two vectors and adding row names
```

```
results <- data.frame("MSE" = MSE,  
                     "RMSE" = RMSE)  
rownames(results) <- c("KNN", "MARS",  
                      "GAM", "LR",  
                      "SVR")
```

```
results
```

```
##           MSE      RMSE  
## KNN  237.2307 15.40230  
## MARS  175.9543 13.26478  
## GAM   182.4715 13.50820  
## LR    182.8450 13.52202  
## SVR   182.4174 13.50620
```

```
kable(results)
```

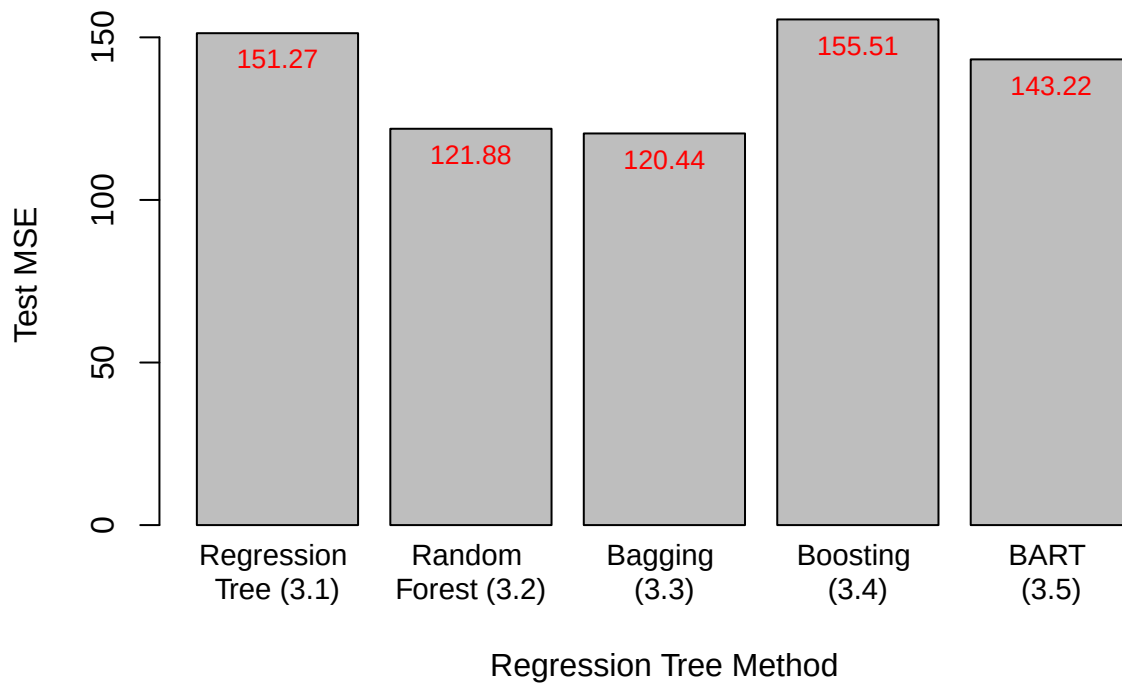
	MSE	RMSE
KNN	237.2307	15.40230
MARS	175.9543	13.26478
GAM	182.4715	13.50820
LR	182.8450	13.52202
SVR	182.4174	13.50620

Regression Tree Models

```
# Combine the error values into a vector
tree_errors <- c(tree_error, rf_error,
                 bag_error, boost_error,
                 bart_error)
names(tree_errors) <- c("Regression \nTree (3.1)",
                       "Random \nForest (3.2)",
                       "Bagging \n(3.3)",
                       "Boosting \n(3.4)",
                       "BART \n(3.5)")

# Creating a bar plot of the errors
bar <- barplot(tree_errors,
               cex.names = 0.9,
               xlab = "Regression Tree Method",
               ylab = "Test MSE",
               main = "Test Errors for Different Regression Tree Methods")
text(x = bar, y = tree_errors, label = round(tree_errors,2),
     pos = 1, cex = 0.8, col = "red")
```

Test Errors for Different Regression Tree Methods



```
# MSE and RMSE values for each regression tree model
tree_MSE <- c(tree_error, rf_error,
              bag_error, boost_error,
              bart_error)
tree_RMSE <- c(tree_rmse, rf_rmse,
              bag_rmse, boost_rmse,
              bart_rmse)

# Combining the two vectors and adding row names
tree_results <- data.frame("MSE" = tree_MSE,
                          "RMSE" = tree_RMSE)
rownames(tree_results) <- c("Regression Tree",
                          "Random Forest",
                          "Bagging",
                          "Boosting",
                          "BART")

tree_results
```

```
##           MSE      RMSE
## Regression Tree 151.2734 12.29933
## Random Forest   121.8815 11.04000
## Bagging         120.4397 10.97450
## Boosting        155.5093 12.47034
## BART            143.2191 11.96742
```

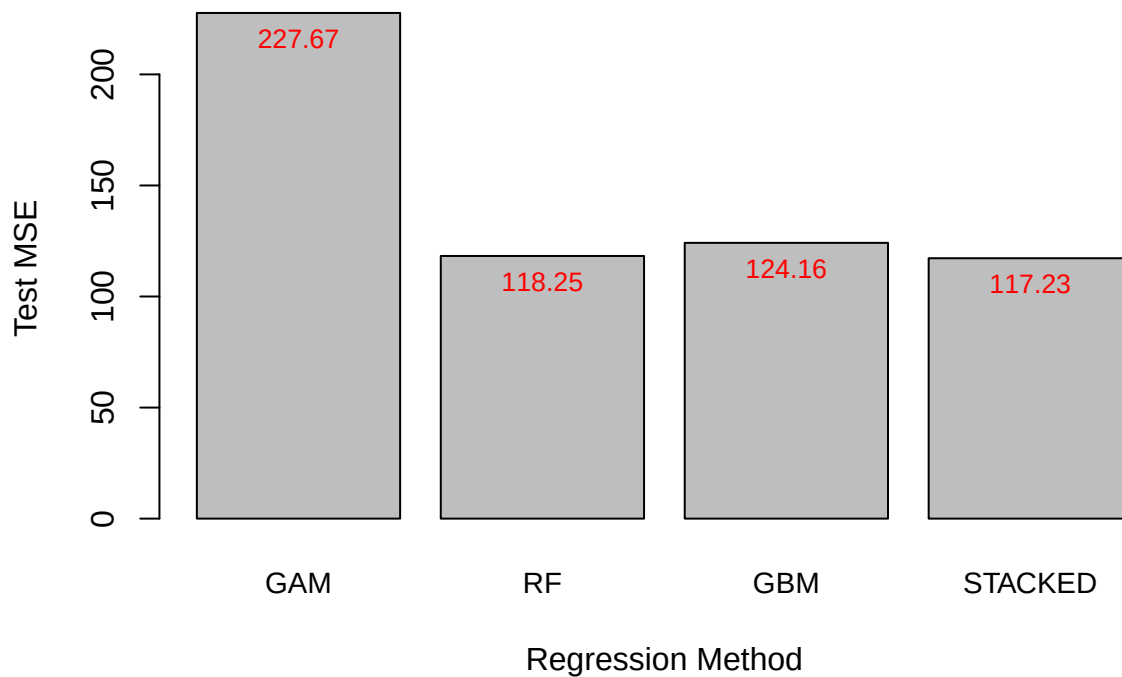
```
kable(tree_results)
```

	MSE	RMSE
Regression Tree	151.2734	12.29933
Random Forest	121.8815	11.04000
Bagging	120.4397	10.97450
Boosting	155.5093	12.47034
BART	143.2191	11.96742

Stacked Model

```
# Creating a bar plot of the errors
bar <- barplot(ensemble_error,
               cex.names = 0.9,
               xlab = "Regression Method",
               ylab = "Test MSE",
               main = "Test Errors for Different Regression Methods & Stacking")
text(x = bar, y = ensemble_error, label = round(ensemble_error,2),
     pos = 1, cex = 0.8, col = "red")
```

Test Errors for Different Regression Methods & Stacking



```
# Combining vectors into a data frame
results <- data.frame("MSE" = ensemble_error,
                      "RMSE" = ensemble_rmse)
results
```

```
##           MSE      RMSE
## GAM      227.6656 15.08859
## RF       118.2482 10.87420
## GBM      124.1598 11.14270
## STACKED  117.2267 10.82713
```

```
kable(results)
```

	MSE	RMSE
GAM	227.6656	15.08859
RF	118.2482	10.87420
GBM	124.1598	11.14270
STACKED	117.2267	10.82713